



Development of a plant sensing robot using virtual space and evaluation of its accuracy

Sutan Muhamad Sadam Awal^{a,b}, Yamato Sakurai^a, Dong Thanh Pham^c, Koichi Nomura^b, Masaharu Kitano^b, Daisuke Yasutake^{c,b}, Muneshi Mitsuoka^c, Takashi Okayasu^{c,b,*}

^a Graduate School of Bioresources and Bioenvironmental Sciences, Kyushu University, Fukuoka, 819-0395, Japan

^b IoP Collaborative Creation Center, Kochi University, Kochi, 780-8520, Japan

^c Faculty of Agriculture, Kyushu University, Fukuoka, 819-0395, Japan

ARTICLE INFO

Keywords:

Omnidirectional agricultural robotics
Gazebo simulation
Robot operating system 2 (ROS2)
LiDAR-based navigation
Fruit tracking and counting

ABSTRACT

Greenhouse robots must perform navigation and fruit sensing under irregular canopy boundaries, occlusion, changing observation conditions, and constrained row-transition spaces. This study developed a simulation-driven ROS 2–Gazebo workflow for evaluating an omnidirectional mobile robot integrating LiDAR-based crop-row navigation, AprilTag-guided multi-row mission control, and ripeness-specific cherry-tomato counting. The system combined LiDAR row centering, PTZ-RGB and front RGB cameras, AprilTag-based mission control, YOLO instance segmentation, ByteTrack, and canonical-identity-based counting. The workflow was evaluated in a procedurally generated greenhouse with heterogeneous plant structures and known plant-level fruit distributions. The AprilTag-guided mission reproduced the intended multi-row route at commanded speeds of 0.2–1.0 m/s, with mean cross-track errors of 0.045–0.056 m and RMSE values below 0.10 m. Fruit-counting performance was strongly affected by robot speed. At 0.2 m/s, the system recovered 90.1% of the ground-truth fruit count with a count-level F1-score of 0.898. Performance declined above 0.4 m/s because of reduced observation time, canopy occlusion, and unstable temporal association. Additional errors arose from canopy obstruction and confusion between orange and red fruits. The proposed workflow enables repeatable evaluation of navigation, mission control, perception, and counting functions before physical deployment. It supports early identification of technical limitations and speed-accuracy trade-offs, thereby reducing trial-and-error, shortening the lead time to real-greenhouse experiments, and decreasing the number and duration of physical tests required for precision-agriculture technology development.

1. Introduction

Agricultural production is increasingly affected by rural labor shortages, aging farming populations, and the declining availability of workers for physically demanding tasks. Globally, agrifood systems employed approximately 1.3 billion people in 2021, while agriculture alone accounted for 892 million workers in 2022 [1]. The effect of automation varies among regions, but robotic systems are increasingly considered important for reducing repetitive labor, improving operational efficiency, and maintaining production under limited labor availability [2,3]. Japan represents a particularly severe case because its farming population is rapidly declining and aging [4]. These conditions have encouraged the development of agricultural robots for monitoring, harvesting, and other field operations [5].

Reliable agricultural robotics remains challenging because crop environments contain irregular plant structures, variable illumination, occlusion, plant motion, and changing layouts. These factors affect both autonomous navigation and vision-based sensing. Simulation-driven development can reduce testing cost and risk by allowing navigation and perception algorithms to be evaluated repeatedly before physical deployment. Robotics simulators provide controlled environments for rapid development, debugging, and reproducible experimentation [6]. Gazebo is widely used because it supports robot dynamics, virtual sensors, and integration with ROS-based control systems [7,8].

ROS–Gazebo simulation has been applied to several agricultural robotics tasks. Li et al. [9] integrated crop-row navigation, multi-camera perception, tracking, and visual SLAM for phenotyping. Betrò et al. [10] investigated GNSS-free LiDAR navigation using structured

* Corresponding author.

E-mail addresses: sutanmsadoc@gmail.com (S.M.S. Awal), yamayuitomo@gmail.com (Y. Sakurai), phamtdong0406@gmail.com (D. Thanh Pham), jm-koichi.nomura@kochi-u.ac.jp (K. Nomura), mkitano@kochi-u.ac.jp (M. Kitano), yasutake@bpes.kyushu-u.ac.jp (D. Yasutake), mitsuoka.muneshi.583@m.kyushu-u.ac.jp (M. Mitsuoka), okayasu@bpes.kyushu-u.ac.jp (T. Okayasu).

<https://doi.org/10.1016/j.atech.2026.102429>

Received 12 May 2026; Received in revised form 20 July 2026; Accepted 21 July 2026

Available online 24 July 2026

2772-3755/© 2026 The Authors. Published by Elsevier B.V. This is an open access article under the CC BY-NC license (<http://creativecommons.org/licenses/by-nc/4.0/>).

vineyard landmarks, while Martini et al. [11] developed a synthetic-data framework for visual navigation in row crops. Iqbal et al. [12] used ROS simulation for LiDAR-based phenotyping and crop-row navigation, and Singh et al. [13] developed a greenhouse digital twin for strawberry detection and robotic manipulation. Other studies have improved simulation realism through plant-motion modeling and high-quality image rendering [14,15]. More recent work has further advanced LiDAR-based inter-row navigation for structured cultivation: Jiang et al. [16] combined 3D LiDAR SLAM with NDT-ICP point-cloud registration for orchard row navigation, and Fujinaga [17] developed a LiDAR-based navigation method for high-bed cultivation that keeps the robot at a constant distance from the bed and was validated in both virtual and real greenhouse environments.

Despite these advances, many existing studies focus mainly on navigation, harvesting, or isolated perception tasks. Fewer studies investigate a complete greenhouse-monitoring workflow that combines mobile navigation, controllable camera viewpoints, fiducial-guided mission control, fruit segmentation, multi-object tracking, and counting within one simulation environment. This integration is important because reliable fruit monitoring depends not only on visual-model accuracy, but also on robot path stability, camera orientation, acquisition speed, occlusion, and temporal association across image frames.

ROS 2 provides a suitable architecture for this integration through its modular communication framework, DDS-based middleware, and Quality of Service mechanisms for heterogeneous sensors [18]. Fiducial markers such as AprilTag can support controlled mission transitions by providing tag identification and relative pose information from monocular images [19]. In greenhouse navigation, such markers can be used to trigger forward and backward row traversal, camera reorientation, lateral row transitions, and mission termination.

Vision-based fruit detection has also advanced through real-time deep-learning models. The YOLO family is widely used in agricultural applications because of its favorable balance between accuracy and inference speed [20]. Recent versions, including YOLOv11, provide improved feature extraction and accuracy–efficiency trade-offs for small and partially occluded objects [21–23]. In particular, instance-segmentation models have recently been tailored to fruit ripeness assessment, including tomato-specific approaches that address occlusion, overlap, and variable illumination in greenhouse and field conditions [24,25]. However, detection or segmentation alone is insufficient for fruit counting from video because the same fruit may appear in multiple frames. ByteTrack addresses this problem by associating detections temporally, including lower-confidence detections that may otherwise interrupt object tracks [26]. Previous agricultural studies have shown that tracking improves counting robustness under occlusion, illumination variation, and viewpoint changes [27–29].

In previous work, a PTZ-camera-based plant phenotyping system was developed using an overhead rail robot for high-resolution cucumber monitoring [30]. The system achieved high YOLOv8 detection performance and demonstrated the benefit of adjustable camera viewpoints. However, the fixed rail structure limited flexibility in dynamic or reconfigurable greenhouse environments. The robotic platforms used in that study are shown in Fig. 1.

In contrast to the fixed-rail platforms in Fig. 1, which constrain motion to a predefined track, the present study adopts a wheeled omnidirectional base to enable flexible, reconfigurable operation; what carries over from the previous work is the concept of active PTZ-camera viewpoint control, not the rail hardware. Building on that work, the present study develops a ground-based omnidirectional mobile robot equipped with 3D LiDAR, a PTZ-RGB camera, and a front RGB camera. The system

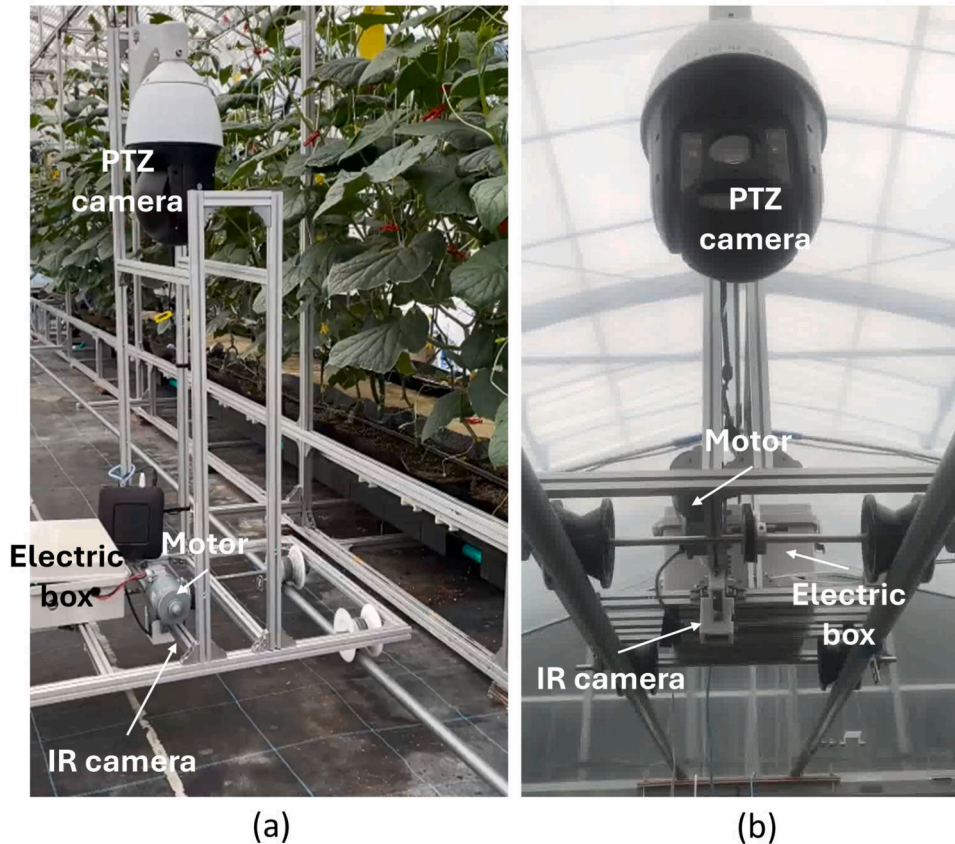


Fig. 1. Robotic platforms from previous work [30], included for motivation: (a) ground-based rail robot and (b) overhead rail robot. These fixed-rail platforms are not used in this study; all experiments reported here were conducted with the wheeled omnidirectional robot described in Section 2.

is implemented in ROS 2 and Gazebo to support repeatable evaluation before physical deployment. LiDAR-based lateral control maintains the robot near the crop-row center, while odometry-based heading control stabilizes its orientation. AprilTags coordinate the complete multi-row mission, including bidirectional row traversal, PTZ-camera reorientation, lateral movement between rows, and mission termination. Fruit perception is performed using YOLOv11 instance segmentation, while ByteTrack is used to maintain fruit identities and reduce repeated counting across frames.

The proposed framework is therefore not limited to fruit detection. It provides an integrated simulation-driven workflow for evaluating how robot navigation, camera viewpoint, acquisition speed, segmentation, tracking, and fruit counting interact during greenhouse monitoring. Procedurally generated plants and fruits provide exact plant-level ground truth, enabling controlled evaluation of different plant distributions and robot operating conditions. The framework also supports separate evaluation of LiDAR row-centering, AprilTag detection, complete multi-row mission execution, model performance, and fruit-counting accuracy.

1.1. Objectives and contributions

The objective of this study was to develop and evaluate a virtual greenhouse platform that supports repeatable, pre-deployment integration and diagnosis of the interactions among robot navigation, mission control, viewpoint selection, segmentation, tracking, and counting. To this end, the study makes three principal contributions:

- An integrated ROS 2–Gazebo simulation workflow that unifies LiDAR-based crop-row centering, AprilTag-guided multi-row mission control with active PTZ viewpoint selection, and a YOLO segmentation, ByteTrack, and canonical-identity counting pipeline into a single reproducible monitoring loop.
- A procedurally generated greenhouse environment with exact plant- and fruit-level ground truth, providing a heterogeneous yet reproducible benchmark for the controlled evaluation of navigation and perception.
- A joint, cross-speed evaluation and diagnosis of navigation accuracy, AprilTag-guided mission behavior, segmentation performance, and fruit-counting consistency, together with an analysis of the simulation-to-real limitations that this virtual evaluation reveals.

By combining mobile navigation, active sensing, fiducial-guided mission control, and temporal fruit perception in one virtual environment, the proposed framework provides a practical basis for developing and de-risking autonomous greenhouse-monitoring robots before real-world deployment.

The remainder of this paper is organized as follows. [Section 2](#) describes the robot platform, simulation environment, navigation framework, and perception methods. [Section 3](#) presents the experimental results and discussion, including an analysis of the simulation-to-real gap and study limitations. Finally, [Section 4](#) summarizes the main findings and outlines directions for future work.

2. Materials and methods

2.1. System and simulation setup

The simulation and control system were implemented on a Linux-based workstation running Ubuntu 22.04.3 LTS (Jammy Jellyfish). The workstation was equipped with an Intel Core i5-13400F CPU, 16 GB RAM, and an NVIDIA GeForce RTX 4070 GPU. Python 3.10.12 was used for implementing the control, data-processing, and experiment-support scripts.

The robotic software framework was developed using ROS2 Humble, while the greenhouse simulation environment was executed in Gazebo Classic version 11.10.2. Gazebo Classic was selected because it provides

stable integration with ROS2 Humble and supports physics-based simulation, URDF/Xacro-based robot models, simulated sensors, and reproducible virtual greenhouse environments. The mobile robot model was described using URDF/Xacro and launched together with the generated greenhouse world through a ROS2 launch system. The simulation pipeline initialized the Gazebo server and client, loaded the virtual greenhouse environment, spawned the robot model, and executed the navigation and perception modules within the same simulated environment.

To ensure reproducibility, the main Gazebo world parameters were explicitly defined. The simulation used the ODE physics engine with a maximum step size of 0.001 s, a real-time factor of 1, and a real-time update rate of 1000 Hz. Gravity was set to 9.81 m/s^2 in the negative vertical direction. The rendering environment used a fixed directional sun-like light source with shadow casting enabled. The light source was configured with a diffuse component of (0.8, 0.8, 0.8, 1) and a specular component of (0.2, 0.2, 0.2, 1), while the scene ambient and background values were set to (0.4, 0.4, 0.4, 1) and (0.7, 0.7, 0.7, 1), respectively. These lighting parameters were kept constant throughout all experiments to provide repeatable visual conditions for navigation and fruit-detection evaluation.

The simulated environment was configured as a controlled virtual greenhouse rather than a fully dynamic outdoor scene. Wind effects were not activated, and no time-varying illumination, weather disturbance, or day–night transition was applied. This configuration was selected to isolate the performance of the robot navigation, LiDAR-based row-following control, and perception pipeline under consistent simulation conditions. The resulting setup enabled repeatable evaluation of the proposed system while maintaining a clear boundary between controlled simulation performance and future real-greenhouse validation. [Table 1](#) show simulation configuration used in the experiments.

2.2. Virtual greenhouse environment generation

To construct a greenhouse environment suitable for evaluating vision-based fruit detection and counting, a cherry tomato plant model with three ripeness levels was adopted: green (unripe), orange (intermediate), and red (ripe). These visual categories provide controlled appearance variation for detector training and evaluation while maintaining a consistent simulation structure. The plant models and greenhouse environment were procedurally generated using the tomato farm simulation framework provided by Espejel et al. [31], which supports the creation of randomized tomato plant configurations and structured greenhouse layouts for robotic simulation studies.

The simulated greenhouse was assembled and evaluated in Gazebo Classic. Instead of manually modeling each plant and greenhouse asset from scratch, the adopted framework was used to generate crop rows, plant placement, fruit-texture categories, and canopy structures in a repeatable virtual environment. The generated world was configured using the following parameters: seed value = 1000, model name prefix = `cherry_tomato`, number of rows = 6, row length = 10 m, inter-row spacing = 1.2 m, intra-crop spacing = 0.5 m, and world size = 9×13 m. These parameters were selected to create a structured greenhouse layout that supports controlled testing of navigation and perception pipelines while remaining reproducible for repeated experiments.

Plant arrangement and fruit distribution were randomized within the constraints of the generator to introduce visual diversity among simulated plants. [Table 2](#) summarizes the main parameters used to generate the virtual greenhouse environment. The use of a fixed seed ensures that the same environment can be regenerated, while the randomized plant configurations provide variation in plant structure, fruit placement, and canopy appearance across the simulated crop rows.

It should be noted that the generated tomato morphology is a simplified representation of real greenhouse tomato plants. Because the adopted generator relies on predefined plant, branch, fruit, and truss templates, it does not fully reproduce the biological variability observed

Table 1
Hardware, software, and Gazebo simulation configuration used in the experiments.

Component	Configuration	Description
Operating system	Ubuntu 22.04.3 LTS	Linux-based simulation workstation
CPU	Intel Core i5-13400F	Main processor
RAM	16 GB	System memory
GPU	NVIDIA GeForce RTX 4070	Graphics acceleration for simulation and visualization
Python version	Python 3.10.12	Control and data-processing scripts
ROS version	ROS2 Humble	Robotic software framework
Simulator	Gazebo Classic 11.10.2	Physics-based virtual greenhouse simulation
Physics engine	ODE	Gazebo physics engine
Maximum step size	0.001 s	Physics simulation time step
Real-time factor	1.0	Target real-time simulation speed
Real-time update rate	1000 Hz	Physics update frequency
Gravity	0, 0, -9.81 m/s ²	Standard gravitational acceleration
Lighting model	Fixed directional light	Sun-like light source with shadows enabled
Diffuse light	(0.8, 0.8, 0.8, 1)	Main light intensity/color component
Specular light	(0.2, 0.2, 0.2, 1)	Reflection component of the light source
Ambient scene light	(0.4, 0.4, 0.4, 1)	Global scene illumination
Background color	(0.7, 0.7, 0.7, 1)	Gazebo scene background
Wind disturbance	Not activated	No dynamic wind effect was applied
Robot model format	URDF/Xacro	Robot description for ROS2 and Gazebo integration

Table 2
Parameters used for generating the simulated tomato plants and greenhouse environment.

Parameter	Value / range	Description
Plant height	0.8–1.2 m	Randomized plant height range
Fruit count per plant	0–36	Random fruit distribution within generator constraints
Fruit color classes	Green, Orange, Red	Ripeness categories for detection and counting
Plant spacing	0.5 m	Distance between plants
Row spacing	1.2 m	Distance between crop rows
Row length	10 m	Length of each crop row
Number of rows	6	Total generated crop rows
World size	9 × 13 m	Gazebo simulation area
Seed value	1000	Reproducibility of generated environment

in actual tomato cultivation. In particular, realistic fruit-size variation, mixed ripeness within individual trusses, vertical fruit-distribution patterns, and progressive maturation along the plant are only partially represented. Therefore, the virtual greenhouse developed in this study should be interpreted as a controlled and reproducible synthetic environment for evaluating robot navigation, sensing, segmentation, and fruit-counting workflows, rather than as a botanically faithful reconstruction of a real tomato greenhouse.

In addition to the plant-arrangement parameters, simplified fruit and leaf textures were used to provide visually distinguishable crop components for perception experiments. As shown in Fig. 2, the fruit models were represented using the same basic 3D template with different surface textures corresponding to the three ripeness levels: green, orange, and red. The leaf model used a tomato-like shape and texture to provide a structured canopy representation for simulated sensing experiments. Fig. 2(a)–(c) show the fruit textures for each ripeness stage, Fig. 2(d) shows the leaf texture, and Fig. 2(e) presents the 3D template used for the tomato and leaf assets in the simulated environment.

To support navigation experiments, a greenhouse structure was added to enclose the crop rows. AprilTags were also placed at predefined locations to serve as reference landmarks for robot localization and navigation events, such as row transitions and sensing triggers. Fig. 3 shows the generated Gazebo greenhouse environment with crop rows and AprilTag placements. Overall, the virtual environment provides a repeatable and controllable platform for testing the integration of ROS2,

Gazebo, autonomous navigation, object detection, and fruit counting under simplified but structured greenhouse conditions.

2.3. Ground-truth fruit distribution statistics for the generated plant community

To characterize the procedurally generated greenhouse environment, descriptive statistics were calculated from the ground-truth plant and fruit distribution. Since the morphological assumptions and simplifications of the plant generator have been described in the previous subsection, the purpose of this analysis is not to claim botanical realism, but to document the numerical distribution of fruits used as the controlled test condition for navigation and perception experiments. The virtual greenhouse consisted of six crop rows, with 21 plant instances in each row, resulting in a total of 126 plants. Each fruit instance was assigned to one of three ripeness classes: green, orange, or red. These statistics were used to quantify the variability introduced by the procedural generator and to confirm that the simulated environment contained heterogeneous fruit-density and ripeness conditions. The generated environment contained 1320 ground-truth fruits in total. Red fruits were the dominant ripeness class, accounting for 828 fruits or 62.7% of the total population. Orange fruits accounted for 336 fruits or 25.5%, while green fruits accounted for 156 fruits or 11.8%. The mean fruit count per plant was 10.48 ± 11.66 , indicating substantial variation among plant instances. In addition, 54 out of 126 plants contained no fruits, corresponding to 42.9% of the plant population. To further quantify the numerical heterogeneity of the generated fruit population, the coefficient of variation of fruit count per plant and the normalized Shannon entropy of the ripeness-class distribution were calculated. The coefficient of variation was selected because it expresses dispersion relative to the mean and provides a dimensionless measure of plant-to-plant variability in fruit abundance. It was calculated as

$$CV = \frac{s_{\text{fruit}}}{\bar{x}_{\text{fruit}}} \cdot 100\%, \quad (1)$$

where $\bar{x}_{\text{fruit}}$ and s_{fruit} denote the mean and standard deviation of fruit count per plant, respectively. Using the observed values of $\bar{x}_{\text{fruit}} = 10.48$ and $s_{\text{fruit}} = 11.66$ fruits per plant, the resulting coefficient of variation was $CV = 111.3\%$. This result shows that the standard deviation exceeded the mean fruit count, reflecting substantial plant-to-plant variation in fruit abundance. This variation is consistent with the coexistence of fruit-dense plants and 54 plants without fruit in the generated community. The diversity and evenness of the three ripeness categories were additionally summarized using Shannon entropy, which quantifies the diversity of a categorical distribution [32]. Because the maximum entropy depends on the number of categories, the value was normalized

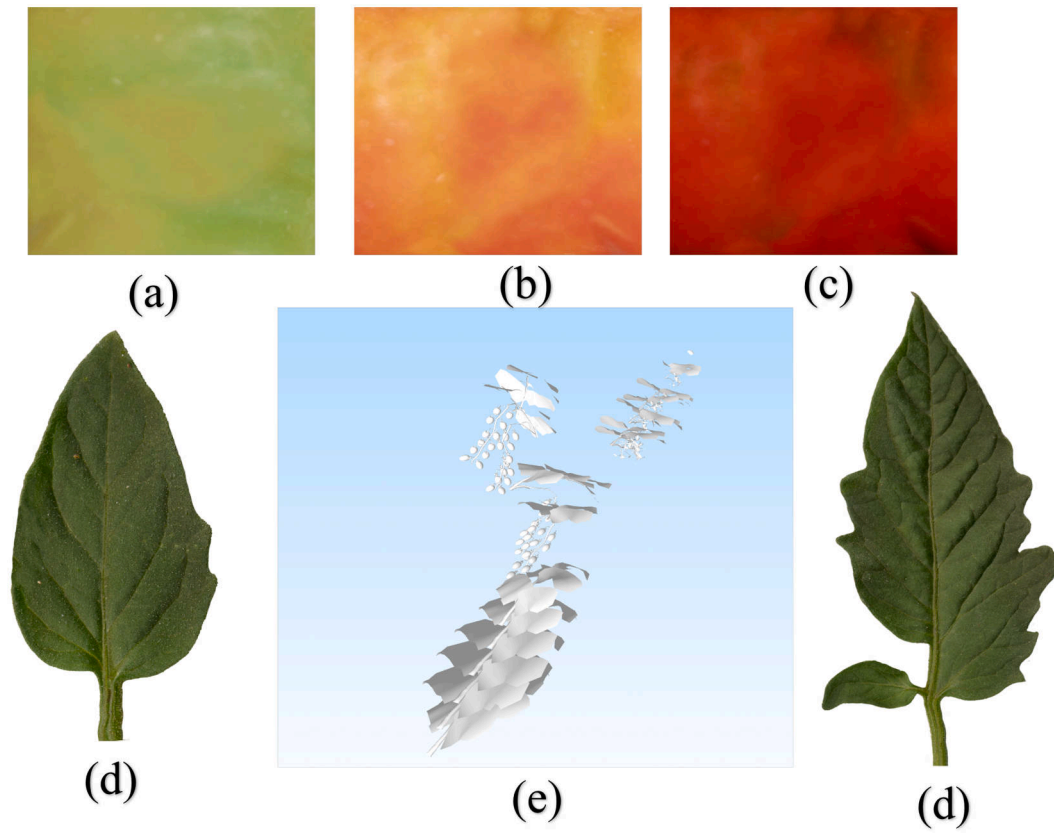


Fig. 2. Visual asset design used in the simulated greenhouse environment. (a) Green tomato texture, (b) orange tomato texture, (c) red tomato texture, (d) leaf texture, and (e) 3D template of the tomato and leaf assets used in the simulated environment. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

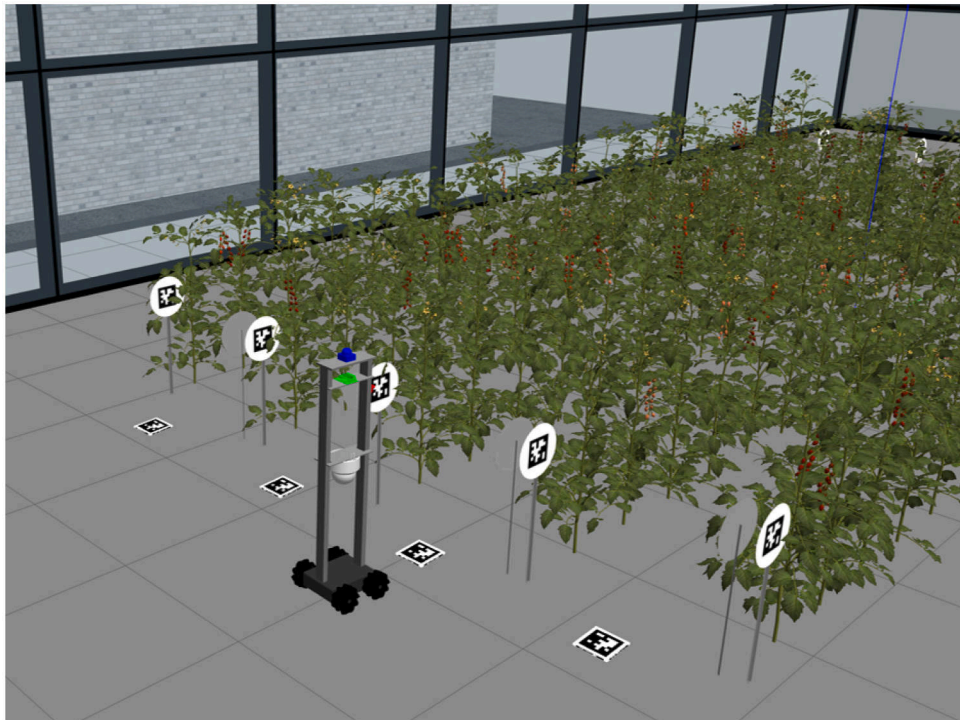


Fig. 3. Generated Gazebo greenhouse simulation with crop rows, greenhouse structure, and AprilTag markers.

by its theoretical maximum, $\ln(K)$, following the normalized evenness formulation described by Pielou [33]:

$$E = \frac{-\sum_{k=1}^K p_k \ln(p_k)}{\ln(K)}, \quad (2)$$

where p_k is the proportion of fruits belonging to ripeness class k , and $K = 3$ represents the green, orange, and red classes. The observed class proportions were calculated as

$$\begin{aligned} p_{\text{green}} &= \frac{156}{1320} = 0.1182, \\ p_{\text{orange}} &= \frac{336}{1320} = 0.2545, \\ p_{\text{red}} &= \frac{828}{1320} = 0.6273. \end{aligned} \quad (3)$$

Using these proportions, the normalized Shannon entropy was calculated as $E = 0.813$. The normalized entropy of 0.81 indicates that all three ripeness categories were represented, although their distribution was uneven and dominated by red fruits. Together, the coefficient of variation and normalized Shannon entropy demonstrate that the generated benchmark was heterogeneous in fruit abundance and ripeness composition. These statistics do not establish complete botanical realism because they do not quantify fruit-size variation, truss morphology, or vertical maturation patterns. However, they provide quantitative evidence that the simulated greenhouse was not a uniform test environment and contained reproducible variation suitable for controlled evaluation of navigation, perception, and fruit-counting performance.

These results indicate that the generated plant community included both fruit-dense and fruit-sparse regions. The observed distribution was therefore treated as part of the ground-truth synthetic testbed and was used to evaluate how the navigation and perception workflow behaved under non-uniform fruit availability.

At the row level, the fruit distribution also showed clear variation. Row 1 contained the lowest number of fruits, with 156 fruits and 14 zero-fruit plants. In contrast, Row 2 and Row 6 contained the highest fruit counts, with 312 and 300 fruits, respectively. The corresponding fruit densities were 14.86 fruits/plant for Row 2 and 14.29 fruits/plant for Row 6. This row-to-row variability reflects the randomized allocation of plant and fruit attributes within the constraints of the procedural generator while preserving the structured row layout required for greenhouse robot navigation experiments.

Fig. 4 shows the ground-truth ripeness distribution aggregated by row. The stacked bars indicate that red fruits dominate the generated fruit population, while green fruits appear less frequently. This class imbalance is reported here as a property of the generated benchmark environment and should be considered when interpreting perception results, especially for class-wise detection performance.

To describe the spatial distribution of fruits at the plant level, Fig. 5 presents the total number of ground-truth fruits assigned to each plant. Each cell represents one plant instance along a crop row. The heatmap shows that fruit density was not uniformly distributed, with dense and sparse plants appearing within the same row. This heterogeneity increases the diversity of visual conditions encountered by the robot during traversal, without implying that the synthetic distribution fully reproduces all biological patterns of real tomato growth.

Fig. 6 further summarizes the dominant ripeness class for each plant. The dominant class was defined as the ripeness category with the largest fruit count on a given plant. Plants without any fruits were marked separately as “No fruit”. This representation provides a compact overview of the spatial arrangement of ripeness classes within the generated greenhouse and helps document the visual variability provided by the synthetic plant community.

Overall, these ground-truth statistics show that the procedural generator produced a reproducible greenhouse test environment with structured row organization, uneven fruit density, ripeness-class imbalance, and plant-level sparsity. These characteristics provide controlled variability for evaluating robot navigation, PTZ-based sensing, fruit segmentation, and counting workflows. However, as noted in the previous subsection, the generated plant community should be interpreted as a controlled synthetic robotic testbed rather than a biologically faithful representation of a real tomato greenhouse.

2.4. Robot modeling and URDF implementation

The mobile robot platform was designed for greenhouse monitoring tasks, including autonomous navigation, plant observation, and fruit-sensing operations using a movable PTZ camera. The mechanical structure was first modeled in Fusion 360 (Autodesk) and then converted into the Unified Robot Description Format (URDF) for integration with ROS 2 and Gazebo. The robot has a compact footprint suitable for greenhouse aisles, with dimensions of 372 mm in length, 374.5 mm in width,

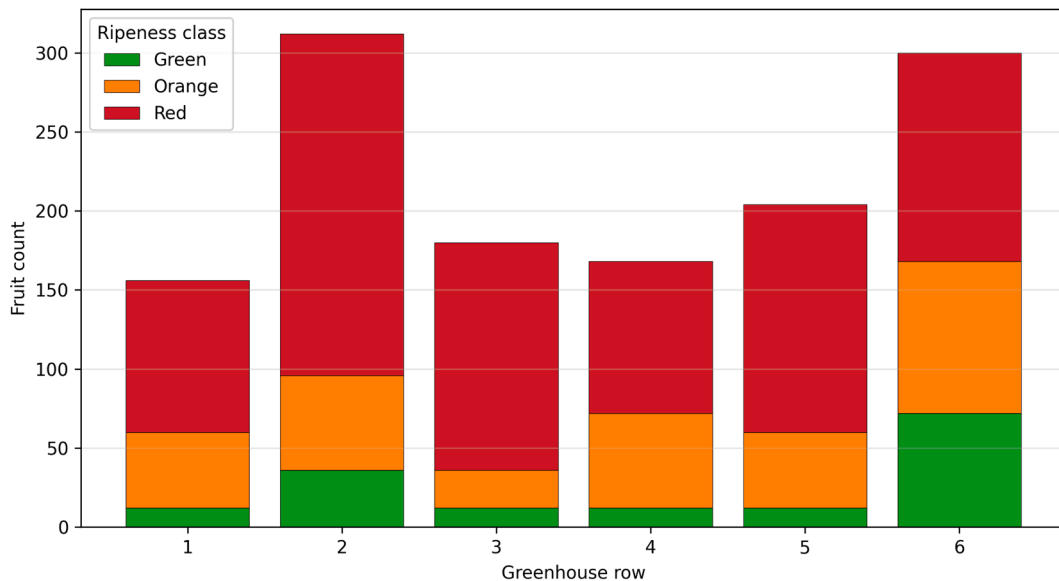


Fig. 4. Ground-truth fruit ripeness distribution per row in the procedurally generated greenhouse environment. Bars show the total number of fruits aggregated across plants in each row and separated by ripeness class.

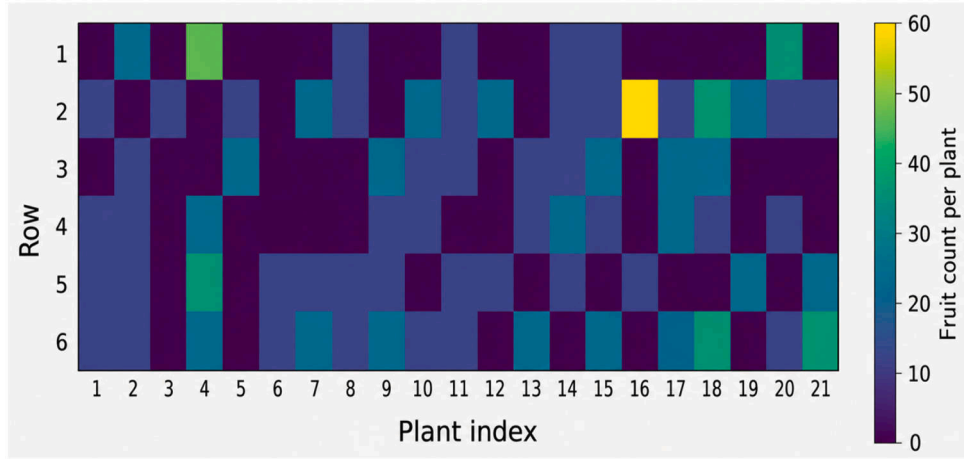


Fig. 5. Ground-truth fruit density per plant in the procedurally generated greenhouse environment. Each cell represents one plant instance indexed along a row, and the value indicates the total number of fruits assigned to that plant.

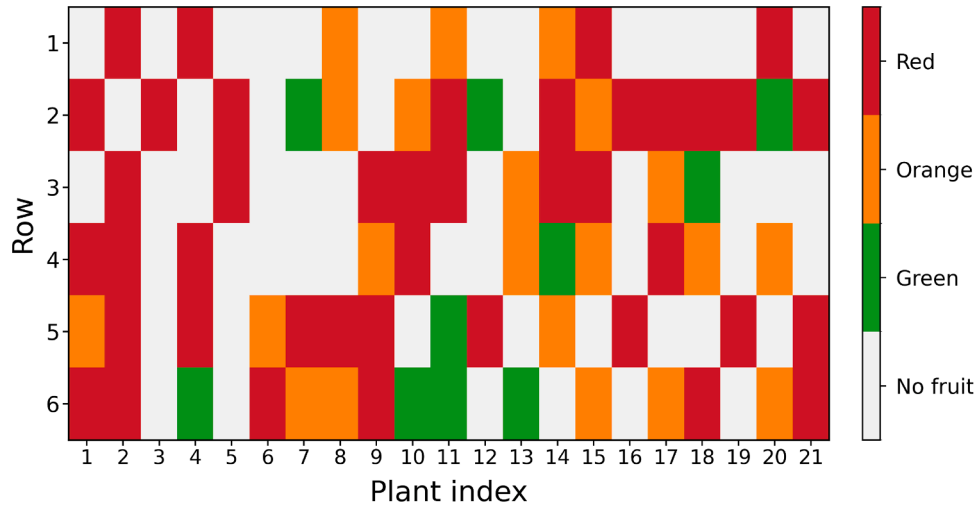


Fig. 6. Dominant ground-truth ripeness class per plant in the procedurally generated greenhouse environment. Colors indicate the ripeness class with the largest fruit count for each plant, while white indicates plants with no fruit.

and approximately 1.4 m in height. The wheelbase, measured as the distance between the front and rear wheel centers, is 283 mm. Fig. 7 presents the CAD model of the developed platform.

The physical robot platform was designed with four Mecanum wheels to support omnidirectional mobility in narrow greenhouse rows. Each wheel has an outer diameter of 152 mm and a width of 55.52 mm, with rollers mounted at a 45° angle. In the Gazebo simulation, however, the wheel joints were represented as fixed visual and structural components because the robot motion was implemented using the `libgazebo_ros_planar_move.so` plugin. This plugin applies planar base motion directly from velocity commands and publishes odometry feedback, allowing the robot to move in the simulated environment without explicitly modeling individual wheel rotation, roller contact, and wheel-ground friction. Therefore, the wheels in the URDF serve primarily to preserve the mechanical appearance and structural layout of the platform, while the planar motion plugin represents the commanded holonomic motion of the mobile base.

This modeling approach was selected because the objective of the simulation was to evaluate greenhouse navigation, LiDAR-based row following, and perception-system integration, rather than to analyze detailed wheel-terrain interaction or Mecanum roller dynamics. By abstracting the low-level wheel contact model, the simulation avoids instability caused by complex roller-ground interactions and provides a re-

peatable platform-level motion response for evaluating navigation and sensing performance. Nevertheless, this simplification is acknowledged as a limitation of the simulation model; future physical validation should include wheel-level actuation, slip effects, and real Mecanum drive dynamics.

The robot structure was represented in URDF using multiple links and joints according to their physical functions. Structural components such as the base frame, wheel assemblies, sensor mounts, and electronic components were connected using fixed joints to preserve the mechanical layout. The camera slider was modeled as a prismatic joint to represent linear height adjustment during sensing. This configuration allowed the simulated robot to maintain the same overall geometry as the CAD model while enabling camera-position adjustment for plant observation tasks.

The URDF model was extended with Gazebo-specific elements to enable physics-based simulation, actuation, and sensor emulation. The `gazebo_ros2_control` plugin was included to connect the simulated robot model with the ROS 2 controller configuration. In addition, the planar motion plugin was used to receive velocity commands and publish odometry information during simulation. This setup enabled the robot to move through the virtual greenhouse using commanded linear and angular velocities while remaining compatible with the ROS 2 control and navigation pipeline.

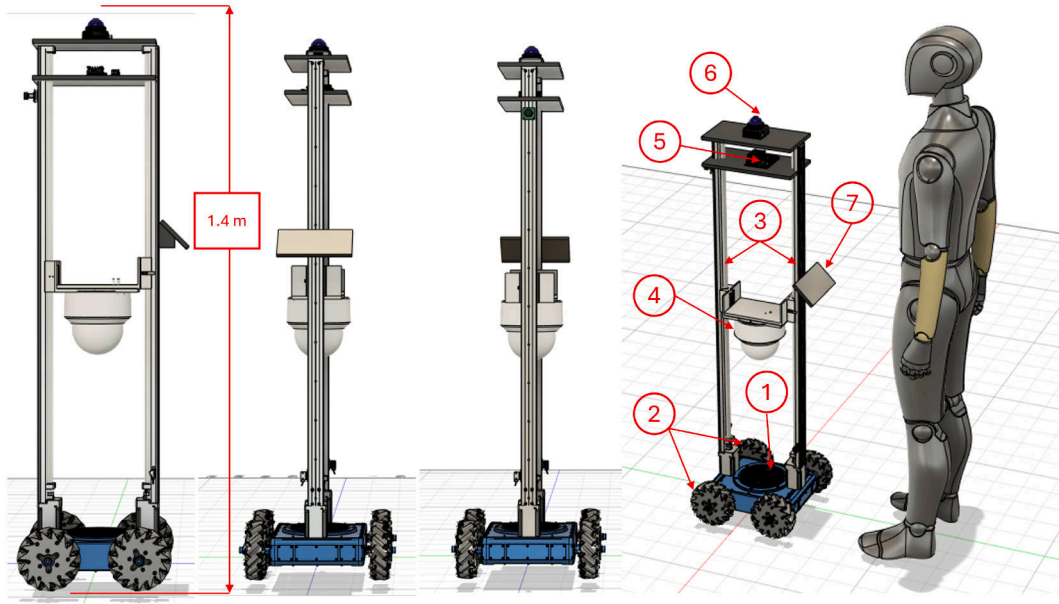


Fig. 7. Design of robot components: 1. base frame, 2. Mecanum wheels, 3. camera slider, 4. PTZ camera, 5. Jetson Orin Nano, 6. 3D LiDAR, 7. LCD touchscreen, and 8. front camera.

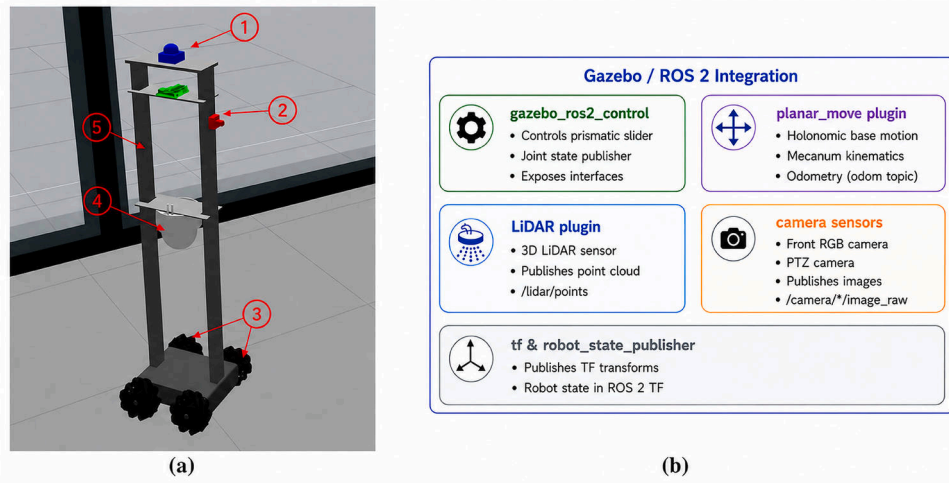


Fig. 8. Robot structure and simulation-system integration. (a) Robot structure in the Gazebo simulation environment: 1. 3D LiDAR, 2. front camera, 3. base platform with Mecanum wheels, 4. PTZ camera, and 5. camera slider. (b) Main Gazebo/ROS 2 integration components implemented in the URDF model, including gazebo_ros2_control, the planar motion plugin, LiDAR plugin, camera sensors, and TF/robot_state_publisher.

The simulated platform was equipped with the main sensing components required for greenhouse monitoring. A 3D LiDAR sensor was mounted on the base platform and configured as a Velodyne-like ray sensor for row perception and obstacle sensing. The sensor was operated at 10 Hz and produced point cloud data used by the row-following controller. Two camera modules were also included in the model. The PTZ camera was configured with a resolution of 1920×1080 pixels and an update rate of 30 Hz for plant observation, while the front RGB camera was configured with a resolution of 800×800 pixels and the same update rate for forward visual perception. Gaussian image noise was added to the simulated camera models to better approximate sensor imperfections in practical conditions.

Overall, the URDF and Gazebo implementation provided a complete simulation representation of the robot platform, including its mechanical structure, holonomic base motion, LiDAR-based perception, and camera-based sensing system. Fig. 8 shows the robot structure in the Gazebo simulation environment, including the principal sensing and actuation components. The main simulation components implemented in the URDF/Gazebo model are summarized in Table 3.

2.5. Robot navigation and localization methods

2.5.1. Navigation system

The navigation system was designed to enable autonomous traversal, bidirectional plant imaging, and row-to-row repositioning within greenhouse crop aisles. The overall navigation sequence is illustrated in Fig. 9. AprilTags are used as discrete mission markers, while a LiDAR-based row-centering controller maintains the robot near the middle of the crop corridor during forward and backward traversal. The LiDAR controller is activated through ROS 2 service calls, whereas the omnidirectional row-transition motion is executed directly using the robot velocity command.

At system initialization, the PTZ camera is oriented toward the right side of the crop row, and the camera slider is fixed at the desired observation height. The PTZ camera and front RGB camera operate simultaneously for AprilTag detection and mission-state transitions.

When PTZ tag 0 is detected within the predefined centroid region, the mission node calls the /move_robot service with SetBool=true. This activates the LiDAR-based navigation controller and initiates for-

Table 3

Main robot components implemented in the URDF/Gazebo simulation model.

Component	Implementation	Purpose
Mecanum wheels	Fixed visual/structural joints	Physical design for omnidirectional mobility; base motion simulated using planar move plugin
Camera slider	Prismatic joint	Adjustable camera height for plant sensing
Planar motion plugin	Gazebo motion interface	Platform-level motion using velocity commands and odometry feedback
ROS2 control plugin	<code>gazebo_ros2_control</code>	Integration with ROS 2 controller configuration
3D LiDAR	Velodyne-like ray sensor, 10 Hz	Row perception and obstacle sensing
PTZ camera	1920 × 1080, 30 Hz	Plant observation and fruit detection
Front RGB camera	800 × 800, 30 Hz	Forward visual perception

ward movement. During this traversal, the controller continuously regulates the robot position using the left–right LiDAR distance measurements, thereby keeping the robot approximately centered within the plant-row aisle. The right-facing PTZ camera acquires images of the plants along the row.

Forward traversal continues until PTZ tag 1 is detected. The mission node then calls the `/stop_robot` service, stops the robot, and rotates the PTZ camera toward the left side of the row. After PTZ tag 2 is detected, the `/move_robot` service is called with `SetBool=false`, activating backward movement under the same LiDAR-based row-centering control. The robot therefore retraces the crop row while acquiring images from the opposite side. Backward traversal terminates when PTZ tag 3 is detected, after which the `/stop_robot` service is called and the row-transition procedure begins.

The row transition is controlled using the front RGB camera and tag 4. The first detected tag 4 represents the current-row transition marker. After this marker is confirmed over multiple frames, the robot moves laterally using its Mecanum-wheel drive. The robot continues the lateral motion until the current tag 4 has remained outside the front-camera view for a predefined interval. A subsequently detected tag 4 is then interpreted as the next-row marker. The robot continues lateral alignment until the centroid of this marker enters the predefined front-camera acceptance region. Once aligned, the robot stops the lateral motion, rotates the PTZ camera toward the right, and moves forward to enter the next crop row.

After entering the next row, PTZ tag 0 is detected again and the same forward–backward imaging sequence is repeated. This combination of AprilTag-based mission transitions, LiDAR-based row-centering, PTZ camera reorientation, and omnidirectional lateral motion enables systematic coverage of multiple greenhouse rows.

Tag 5 is used as the final mission marker. When tag 5 is detected, the mission node calls the `/stop_robot` service and terminates the autonomous navigation sequence. This prevents further row transitions after the final designated crop row has been completed.

2.5.2. LiDAR-based navigation control

The proposed controller uses LiDAR point-cloud data and odometry feedback to maintain the robot near the center of a greenhouse crop row. Because the mobile platform is omnidirectional, lateral row-centering is controlled through `linear.y`, while heading stability is controlled independently through `angular.z`. The controller therefore combines longitudinal motion, lateral correction, and heading hold rather than relying only on yaw adjustment.

The node subscribes to `/points_raw` and `/odom`, and publishes velocity commands to `/cmd_vel`. Each LiDAR point (x, y, z) is retained when

$$z_{\min} < z < z_{\max}, \quad \sqrt{x^2 + y^2} < d_{\max}, \quad |y| > y_{\min}, \quad x > x_{\min}, \quad (4)$$

where $z_{\min} = 0.0$ m, $z_{\max} = 1.0$ m, $d_{\max} = 1.0$ m, $y_{\min} = 0.1$ m, and $x_{\min} = -1.0$ m. These conditions remove ground-level noise, distant points, measurements near the robot centerline, and irrelevant points behind the platform. Points with $y > 0$ and $y < 0$ are assigned to the left and right crop-row boundaries, respectively.

To reduce sparsity, the filtered points are interpolated at a resolution of 0.1 m. The row-boundary distances are then calculated from the

lateral coordinate $|y|$:

$$D_l = \frac{1}{N_l} \sum_{i=1}^{N_l} |y_{l,i}|, \quad D_r = \frac{1}{N_r} \sum_{i=1}^{N_r} |y_{r,i}|, \quad (5)$$

where N_l and N_r are the numbers of valid points on the left and right sides. The last ten valid measurements are stored, and their median values are used to suppress point-cloud fluctuations. A measurement is accepted when it is greater than 0.01 m and differs from the preceding valid measurement by less than 0.5 m.

The lateral centering error is defined as

$$e_s = \bar{D}_l - \bar{D}_r, \quad (6)$$

where $\bar{D}_l$ and $\bar{D}_r$ are the median-filtered distances. A positive value indicates that the robot is closer to the right boundary and should move laterally toward the left.

A discrete PID controller generates the lateral velocity command. A deadband of 0.03 m is applied to prevent unnecessary oscillation near the row center. When $|e_s| < 0.03$ m, v_y is set to zero and the integral term is reset. Otherwise,

$$v_y = \text{sat}[-v_{y,\max}, v_{y,\max}] \left(k_{p,s} e_s + k_{i,s} \sum e_s + k_{d,s} (e_s - e_{s,\text{prev}}) \right), \quad (7)$$

where $v_{y,\max} = 0.12$ m/s, $k_{p,s} = 0.70$, $k_{i,s} = 0.00$, and $k_{d,s} = 0.15$.

Heading stability is maintained using odometry feedback. The initial yaw angle is stored as the reference heading ψ_{ref} , and the heading error is

$$e_\psi = \text{wrap}(\psi_{\text{ref}} - \psi), \quad (8)$$

where $\text{wrap}(\cdot)$ constrains the error to $[-\pi, \pi]$. With a heading deadband of 2° , the angular velocity is calculated as

$$\omega_z = \text{sat}[-\omega_{\max}, \omega_{\max}] \left(k_{p,\psi} e_\psi + k_{i,\psi} \sum e_\psi + k_{d,\psi} (e_\psi - e_{\psi,\text{prev}}) \right), \quad (9)$$

where $\omega_{\max} = 0.25$ rad/s, $k_{p,\psi} = 1.20$, $k_{i,\psi} = 0.00$, and $k_{d,\psi} = 0.15$.

The final command is

$$\mathbf{V}_c = \begin{bmatrix} v_x \\ v_y \\ \omega_z \end{bmatrix}, \quad (10)$$

where $v_x = 0.3$ m/s is the nominal longitudinal velocity, v_y is the LiDAR-based lateral correction, and ω_z is the odometry-based heading correction. The command is published at 10 Hz, while the filtered point cloud is published at 5 Hz.

The controller parameters were tuned empirically in Gazebo. The integral gains were set to zero to avoid wind-up, the proportional gains were increased until adequate centering and heading responses were obtained, and derivative terms were introduced to reduce oscillation caused by irregular crop boundaries and point-cloud discontinuities. The selected parameters provided a compromise between rapid correction and smooth robot motion. Based on these observations, the final parameters shown in Table 4.

The node additionally publishes the filtered cloud to `/filtered_points`, the lateral error to `/side_error`, and the heading error to `/heading_error`. Service interfaces are provided to start forward or backward LiDAR-guided motion, stop the robot, and reset the heading reference. Fig. 10 shows the LiDAR perception output and simulated greenhouse environment used for row-following evaluation.

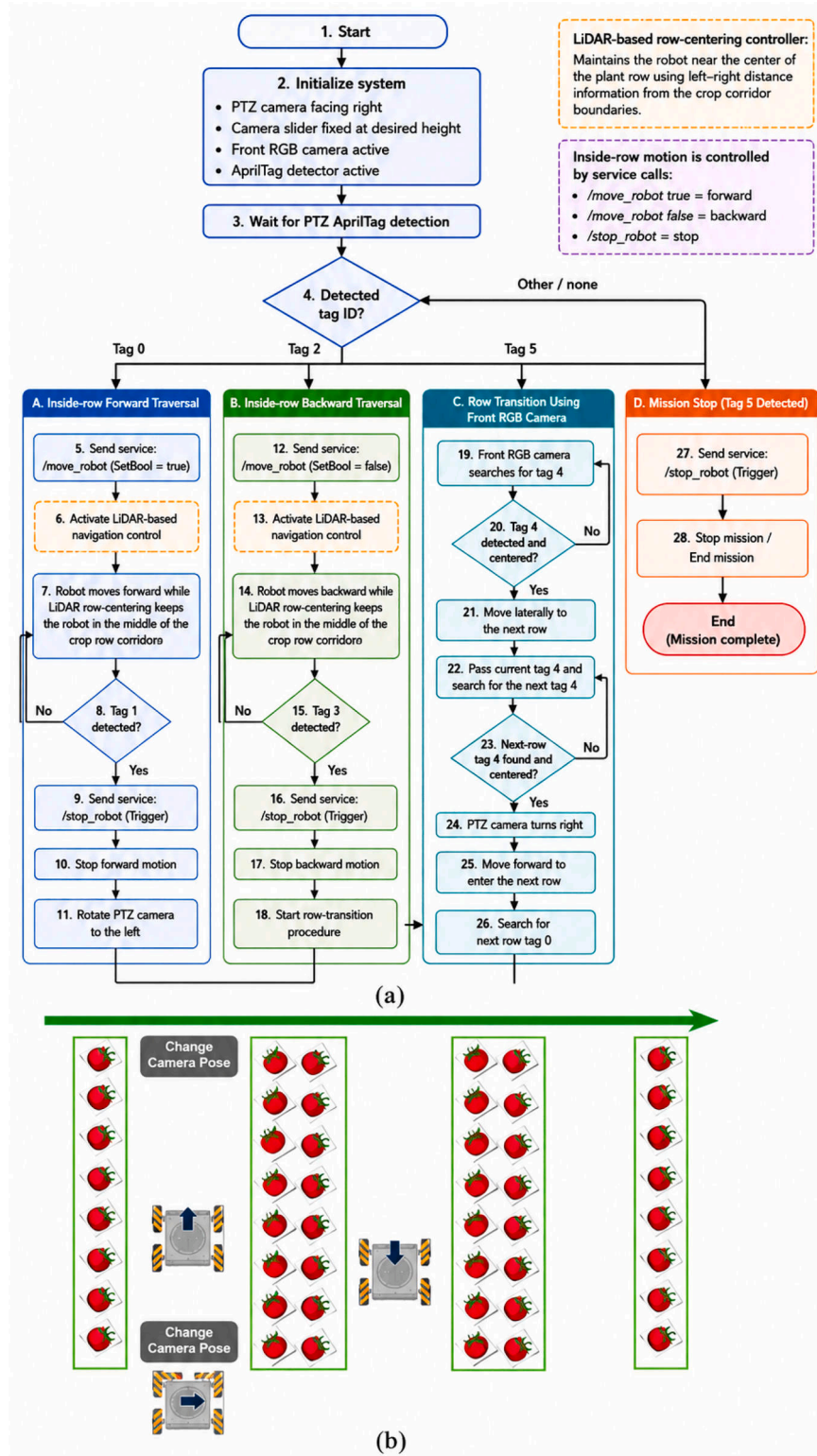


Fig. 9. AprilTag-guided greenhouse navigation workflow and corresponding robot motion sequence. Tags 0–3 control bidirectional in-row traversal through LiDAR-based row-centering services, tag 4 guides lateral row transition using the front RGB camera, and tag 5 terminates the mission.

2.5.3. AprilTag-based navigation markers

AprilTags were used as visual landmarks to identify the robot's position within the navigation sequence and trigger predefined mission transitions. Representative detections are shown in Fig. 11. Tag 0 initiates forward in-row traversal. Tag 1 indicates the end of forward traversal

and triggers PTZ camera reorientation to the left. Tag 2 confirms the starting position for backward traversal, whereas Tag 3 marks the end of backward movement, after which the robot continues moving backward until Tag 4 is detected. Tag 4 is detected using the front RGB camera and provides the reference required for lateral alignment with

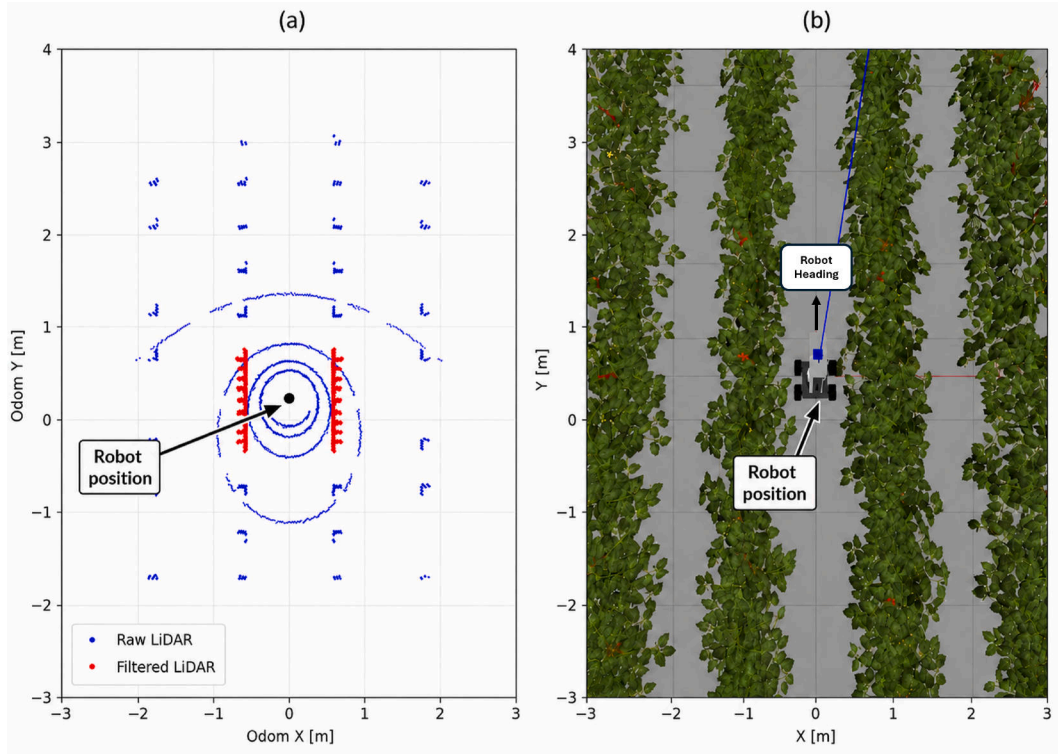


Fig. 10. LiDAR-based row-following perception in the simulated greenhouse: (a) raw point cloud (blue) and filtered points (red) used to estimate the crop-row boundaries and lateral centering error; and (b) simulated robot navigation between tomato rows. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 4

LiDAR-based row-following and heading-control parameters.

Parameter	Value	Description
Forward velocity, v_x	0.3 m/s	Nominal row-following speed
Maximum lateral velocity, $v_{y,max}$	0.12 m/s	Lateral-command saturation limit
Maximum angular velocity, ω_{max}	0.25 rad/s	Heading-command saturation limit
Side k_p, k_I, k_D	0.70, 0.00, 0.15	Lateral row-centering gains
Side deadband	0.03 m	Threshold for zero lateral correction
Heading k_p, k_I, k_D	1.20, 0.00, 0.15	Heading-hold gains
Heading deadband	2°	Threshold for zero angular correction
LiDAR height range	0.0–1.0 m	Accepted vertical range
Maximum point distance	1.0 m	Maximum accepted radial distance
Minimum side distance	0.1 m	Minimum accepted value of $ y $
Minimum forward coordinate	−1.0 m	Removes points far behind the robot
Distance history length	10 samples	Median-filtering window
Command frequency	10 Hz	Velocity-command publication rate
Filtered-cloud frequency	5 Hz	Filtered-cloud publication rate

the next crop row. In addition to the tag ID, the detected centroid and estimated distance are used to verify that each marker is within the pre-defined acceptance region before a navigation transition is executed.

2.5.4. AprilTag detection evaluation setup

The reliability of the AprilTag detection and pose-estimation pipeline was evaluated in Gazebo under three conditions: camera-to-tag distance, viewing angle, and robot speed. A single AprilTag was placed at a known fixed position, while robot odometry, commanded velocity, camera pose, estimated tag pose, camera-to-tag distance, and viewing angle were recorded at 10 Hz.

A sample was considered successful when a valid AprilTag pose estimate was available within the specified timeout. The detection success rate was calculated as

$$S = \frac{N_{\text{valid}}}{N_{\text{total}}} \cdot 100\%, \quad (11)$$

where N_{valid} is the number of samples with a valid pose estimate and N_{total} is the total number of samples in each evaluation bin.

For the distance evaluation, the robot was positioned at different camera-to-tag distances while keeping the marker approximately in front of the camera. Ground-truth distance was calculated from the known tag position and the camera position derived from odometry. The samples were grouped into 0.5 m intervals to determine the effective detection range.

For the viewing-angle evaluation, the robot was placed at different orientations while maintaining a relatively short distance from the tag. Because the camera faced the right side of the robot, its optical direction was represented by a yaw offset of -90° relative to the robot heading. The viewing angle was calculated as the absolute angular difference between the camera optical direction and the camera-to-tag bearing. Samples were grouped into angular intervals, and the success rate and mean distance were calculated for each interval.

For the speed evaluation, the robot passed the tag at several commanded linear speeds. Forward and reverse velocities were converted to their absolute values, and nearly stationary samples with $|v_x| \leq 0.05$ m/s were excluded. For each speed level, the detection success rate, mean camera-to-tag distance, and mean viewing angle were reported.

This procedure evaluates AprilTag performance under both static geometric variation and dynamic robot motion, providing the operating range within which tag-based navigation support can be used reliably in the simulated greenhouse.

2.5.5. Navigation test setup and evaluation

The proposed LiDAR-based row-centering system was evaluated in five greenhouse crop rows and one wall-corridor reference scenario. The wall corridor provided regular boundaries for comparison with the irregular geometry of plant rows. In each environment, the robot traversed an approximately 10 m segment in both forward and backward directions.

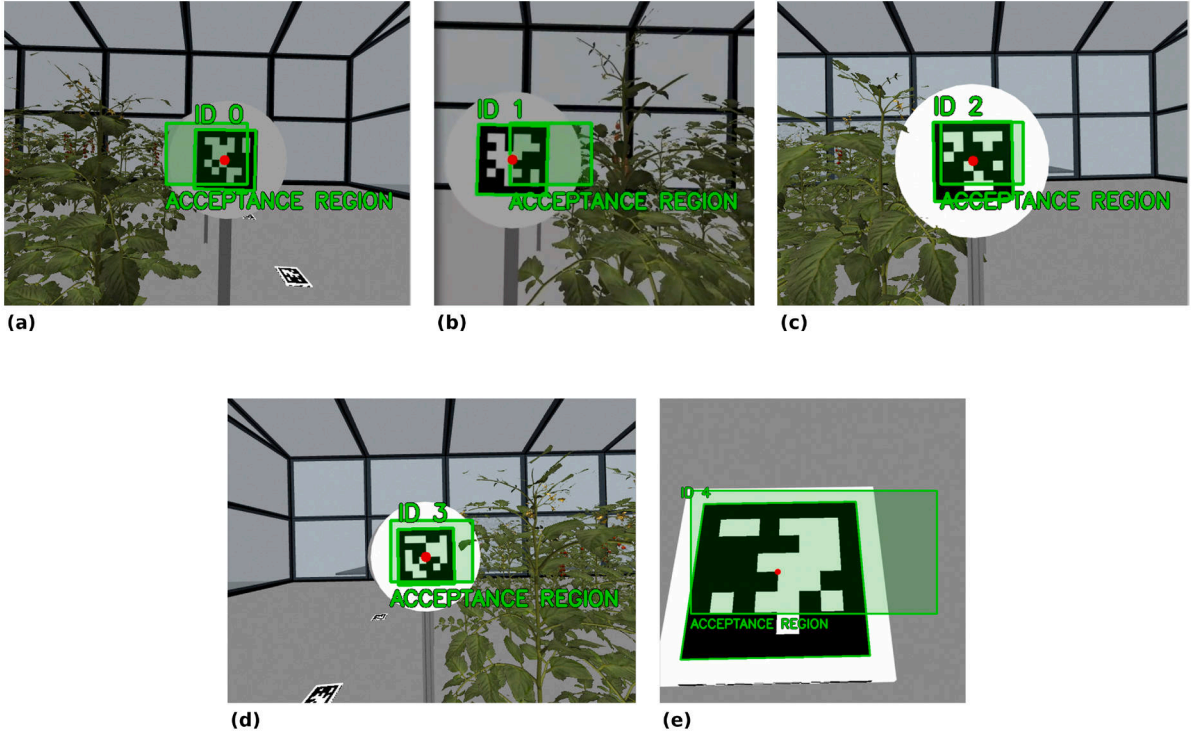


Fig. 11. Representative AprilTag detections used in the navigation workflow. Tag 0 starts the forward in-row traversal. (a) Tag 1 stops the forward traversal and triggers reorientation of the PTZ camera to the left; (b) Tag 2 starts the backward in-row traversal; (c) Tag 3 marks the end of the row, after which the robot continues moving backward until Tag 4 is detected; and (d) Tag 4 guides the lateral row transition to the next row.

For each row and direction, one ground-truth (GT) trajectory and three autonomous repetitions were recorded. The GT path was obtained by manually driving the robot along the desired centerline using a joystick, while the autonomous trials were conducted under the same initial and final conditions using the proposed controller. This design enabled both trajectory accuracy and run-to-run repeatability to be assessed.

During each trial, robot position, yaw angle, commanded and odometry-based velocities, side-centering error, and heading error were recorded. Before evaluation, invalid pose samples and large discontinuities were removed, stationary sections at the beginning and end of each run were trimmed, and the position and yaw signals were smoothed.

To enable a fair comparison, each autonomous trajectory was aligned to the corresponding GT trajectory using its start and end points. Both paths were then resampled into the same number of equally spaced points along normalized path progress. This reduced the influence of differences in trial duration and stopping position and allowed the comparison to focus on path shape. Let $\mathbf{p}_{a,i} = [x_{a,i}, y_{a,i}]^T$ and $\mathbf{p}_{g,i} = [x_{g,i}, y_{g,i}]^T$ denote corresponding autonomous and GT trajectory points. The point-wise position error was

$$e_i = \|\mathbf{p}_{a,i} - \mathbf{p}_{g,i}\|_2. \quad (12)$$

Trajectory performance was evaluated using

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N e_i, \quad (13)$$

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2}, \quad (14)$$

and

$$e_{\max} = \max(e_i), \quad (15)$$

where N is the number of normalized trajectory samples. MAE represents the average path deviation, RMSE emphasizes larger deviations,

and $e_{\max}$ gives the largest local error.

$$\text{Heading MAE} = \frac{1}{N} \sum_{i=1}^N |e_{\psi,i}|. \quad (16)$$

The metrics were calculated separately for each autonomous repetition and then reported as the mean $\pm$ standard deviation over the three runs. The mean values describe tracking performance relative to the GT path, while the standard deviation represents run-to-run variability. This evaluation therefore quantified trajectory accuracy, path repeatability, and heading stability for forward and backward traversal in both structured and crop-row environments.

2.6. Object detection and counting method

2.6.1. Dataset preparation and model training

The dataset used for model training was generated from Gazebo-based simulated greenhouse environments created using two procedural-generation seeds, 1001 and 1002. These environments provided different plant arrangements, fruit locations, and ripeness distributions. The images were manually annotated using Roboflow and exported in YOLO segmentation format. Each fruit instance was labeled into one of three ripeness classes: *cherry_tomato_green*, *cherry_tomato_orange*, or *cherry_tomato_red*. The dataset was split into 70% training, 20% validation, and 10% testing subsets. Fig. 12 presents a representative example of the annotated simulated dataset, illustrating how individual cherry tomato fruits were labeled at the instance level and assigned to the three ripeness classes.

Two dataset configurations were prepared for model comparison. The original non-augmented configuration contained 755 images, comprising 529 training images, 151 validation images, and 75 test images. The dataset partition was completed before augmentation to prevent augmented derivatives of the same image from appearing in different subsets. Augmentation was applied exclusively to the 529-image training subset and generated three training outputs per original image,

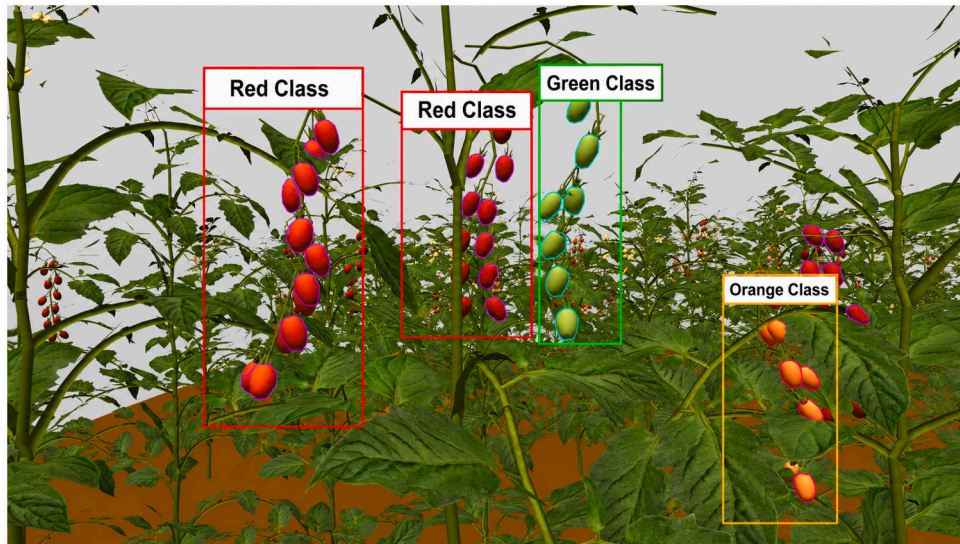


Fig. 12. Representative annotated image from the simulated greenhouse dataset. Each visible cherry tomato fruit was labeled as an individual instance and assigned to one of three ripeness classes: red, green, or orange. The example illustrates the segmentation-based annotation format used for training and evaluating the ripeness-class fruit segmentation models. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

resulting in 1587 training images. The validation and test subsets remained unchanged; therefore, the augmented configuration contained 1813 images in total. The applied transformations included brightness variation between -21% and $+21\%$ and random noise affecting up to 1.41% of pixels. All images were resized to 640×640 pixels.

The dataset contained an imbalanced distribution of ripeness classes, with red fruits appearing more frequently than orange and green fruits. This imbalance occurred because the procedural plant generator did not directly control the exact number of fruit instances produced for each ripeness class. Instead, fruit placement and ripeness distribution were generated within the constraints of the synthetic environment. Therefore, the imbalance was retained as part of the generated simulation condition and was considered when interpreting model performance.

YOLOv8-seg and YOLOv11-seg models were trained and compared using three model scales: nano (n), small (s), and medium (m). Each model was trained using both the non-augmented and augmented dataset configurations, resulting in twelve training runs. Training was conducted on a Windows PC equipped with an Intel Core i5-13400F CPU, 16 GB RAM, and an NVIDIA GeForce RTX 4070 GPU. The models were trained for up to 300 epochs with early stopping, using a patience value of 50, a batch size of 32, stochastic gradient descent (SGD), an initial learning rate of 0.01, and an input resolution of 640×640 pixels.

The objective of this training stage was to select a suitable base model for ripeness-class fruit segmentation before the subsequent fruit-counting evaluation. Model performance was evaluated using mask precision, recall, F1-score, mAP50, and mAP50–95. Since the downstream counting workflow depends on stable instance-level segmentation, mask mAP50–95 was used as the primary segmentation metric, while precision, recall, and F1-score were also considered to account for false-positive and missed-fruit behavior.

It should be noted that the dataset was generated from simulated greenhouse environments and was used primarily to validate the proposed robotic perception–navigation workflow. The trained model served as a functional perception component for evaluating fruit detection, ripeness classification, and counting in simulation. Future transfer to real greenhouse conditions will require additional real-world training data and domain adaptation to account for lighting variation, plant diversity, camera noise, and fruit morphology.

2.6.2. Evaluation criteria for model selection

The trained YOLO segmentation models were evaluated to select a base model for the subsequent fruit-counting experiment. Model selec-

tion was performed using the validation split, while the held-out test split was reserved for final evaluation. Following common object detection and instance segmentation evaluation practice [34], the comparison used mask precision, mask recall, mask F1-score, mask mAP50–95, and bounding-box mAP50–95. Mask mAP50–95 was used as the primary segmentation-quality metric because it averages performance across multiple IoU thresholds, while precision, recall, and F1-score were used to assess false-positive and missed-fruit behavior that could affect counting accuracy [35].

In addition to segmentation accuracy, video-processing speed was evaluated using robot-camera video sequences. Mean inference time per frame and pipeline FPS were reported to assess the practical suitability of each model for robot-based perception. Since the subsequent counting workflow uses tracking-by-detection, stable and balanced detections are important for maintaining object continuity across frames [36]. Therefore, the final model was selected based on the overall balance between segmentation quality, precision–recall behavior, localization performance, and processing speed, rather than the highest mAP value alone.

2.6.3. Fruit counting test setup and evaluation

Fruit counting was evaluated using recorded robot-camera image sequences from all six crop rows in the generated greenhouse environment. Because the plant community was procedurally generated, each row contained different fruit distributions and visual conditions. Therefore, evaluating all rows allowed the counting workflow to be assessed across the complete generated environment rather than using only selected sample rows. For each row, image sequences were acquired under one static condition and five robot traveling speeds: 0.2, 0.4, 0.6, 0.8, and 1.0 m/s. Thus, the evaluation covered six acquisition conditions across six greenhouse rows.

The selected segmentation model was applied to each sequence to detect fruit instances and assign ripeness classes. A tracking-by-detection strategy based on ByteTrack was then used to associate detections across consecutive frames and reduce repeated counting of the same fruit. Fruit counts were updated based on tracked identities rather than per-frame detections. Specifically, a fruit was counted only when the centroid of its tracked instance entered a predefined counting window, and each track was counted at most once. To further reduce repeated counting caused by raw-ID fragmentation, a canonical-ID layer was used to preserve the counted state when a fruit track was re-associated after a temporary tracking interruption. The same segmentation model, tracking



Fig. 13. Illustration of the fruit-counting process in the image plane. (a) Before counting, detected fruits are tracked outside the predefined counting region and remain in the waiting state. (b) When the centroid of a tracked fruit enters the counting region, the count is incremented and the track status changes to counted. (c) After passing through the counting region, the same tracked fruit remains marked as already counted and is not counted again, thereby reducing duplicate counting across consecutive frames.

parameters, and counting-window settings were applied to all evaluated sequences. Because the evaluation was performed from recorded sequences, reprocessing the same sequence was not treated as an independent repeated trial. Instead, the six greenhouse rows were treated as row-level evaluation units, and the results were summarized across rows for each acquisition condition.

The counting process is illustrated in Fig. 13. In Fig. 13(a), detected fruits are tracked before entering the predefined counting region and are not yet counted. In Fig. 13(b), the fruit count is incremented when the tracked centroid enters the counting region. In Fig. 13(c), the same tracked fruits remain marked as already counted after leaving the counting region, so they are not counted again in subsequent frames.

Ground-truth counts were obtained from manual row-plant fruit-count mapping of the generated environment. The ground truth contained fruit counts for each row, plant, and ripeness class. Counting performance was evaluated by comparing the predicted fruit counts with the corresponding ground-truth counts at the plant level. For each evaluated plant, the total predicted count was denoted by C_i and the corresponding ground-truth count by G_i . Plant-level counting accuracy was quantified using the mean absolute error (MAE) and root mean square error (RMSE) defined in Eqs. (13) and (14), with the per-plant error taken as the count difference $C_i - G_i$ between the predicted and ground-truth counts, and N denoting the number of evaluated plant samples. In addition, the proportion of recovered fruits was reported as Pred./GT (%), defined as

$$\text{Pred./GT(\%)} = \frac{\sum_i C_i}{\sum_i G_i} \cdot 100. \quad (17)$$

Count-level precision, recall, and F1-score were computed by interpreting matched counts as true positives, overcounted fruits as false positives, and missed fruits as false negatives:

$$TP_i = \min(C_i, G_i), \quad (18)$$

$$FP_i = \max(C_i - G_i, 0), \quad (19)$$

$$FN_i = \max(G_i - C_i, 0), \quad (20)$$

$$\text{Precision} = \frac{TP}{TP + FP}, \quad (21)$$

$$\text{Recall} = \frac{TP}{TP + FN}, \quad (22)$$

$$F1 = \frac{2 \cdot \text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}, \quad (23)$$

where TP , FP , and FN were accumulated across the evaluated plants for each acquisition condition. These count-level metrics were used to quantify both overcounting and undercounting behavior.

To examine whether acquisition condition significantly affected counting performance, plant-level absolute counting errors were compared using the Friedman test, followed by Holm-adjusted pairwise comparisons when a significant effect was detected [37,38]. A limited pilot

evaluation of ByteTrack was also conducted using eight manually annotated frames from one representative sequence at 0.2 m/s, yielding a MOTA of 34.9%, IDF1 of 54.0%, precision of 89.2%, and recall of 82.8%. Because this pilot used only a small temporally subsampled set of frames, it was treated as a diagnostic assessment rather than a formal MOT benchmark. Therefore, the principal evaluation focused on plant-level MAE, RMSE, precision, recall, F1-score, and statistical comparison of counting errors, while a canonical-identity layer was used to reduce duplicate counting after temporary tracking interruptions.

3. Results and discussion

3.1. AprilTag detection evaluation

The AprilTag detection module was evaluated under three conditions: camera-to-tag distance, viewing angle, and robot speed. A single AprilTag marker was placed at a known fixed position in the simulation environment, and the robot state, camera-to-tag geometry, command velocity, odometry velocity, and detection result were recorded during each test. A detection was considered successful when the target AprilTag was detected and a valid pose estimate was available in the recorded frame. The detection success rate S was computed as defined in Eq. (11), with N_{valid} the number of successful detections and N_{total} the total number of recorded frames in each evaluation condition.

For the distance evaluation, the robot was positioned at different camera-to-tag distances while maintaining the tag approximately within the camera view. The recorded samples were grouped into distance intervals, and the success rate was calculated for each interval. For the viewing-angle evaluation, the robot pose was varied relative to the tag while the camera-to-tag distance was also recorded. Since the camera was mounted on the right side of the robot, the viewing angle was calculated based on the corrected camera heading rather than the robot heading. For the speed evaluation, the robot was commanded to move at different linear speeds while observing the AprilTag. Since the robot could move in both forward and reverse directions, the signed commanded velocity was converted into absolute speed. Frames recorded while the robot was stopped or not moving within the intended speed interval were excluded from the speed-based calculation.

Table 5 summarizes the AprilTag detection performance for the three evaluation conditions. In the distance evaluation, no successful detection was obtained at the closest range of 0.00–0.50 m. At such a short range, the tag is too close to the camera and extends beyond the image frame, so its complete border and all four corners are not captured; because AprilTag decoding requires the full marker boundary, no valid pose can be recovered. However, once the distance increased beyond 0.50 m, the detection performance improved substantially. The success rate reached 98.70% at 0.50–0.75 m and 100% for all tested distance intervals from 0.75 m to 1.50 m. This result indicates that the AprilTag

Table 5

AprilTag detection success rate under distance, viewing-angle, and speed evaluation conditions. For the speed evaluation, signed velocity was converted to absolute velocity, and stopped frames were excluded from the analysis.

Evaluation	Condition	Valid/total	Success rate	Mean distance	Mean angle
Distance	0.00–0.50 m	0/65	0.00%	0.339 m	–
Distance	0.50–0.75 m	76/77	98.70%	0.600 m	–
Distance	0.75–1.00 m	59/59	100.00%	0.845 m	–
Distance	1.00–1.25 m	22/22	100.00%	1.115 m	–
Distance	1.25–1.50 m	57/57	100.00%	1.287 m	–
Viewing angle	0–5°	79/197	40.10%	1.061 m	3.87°
Viewing angle	5–15°	50/50	100.00%	0.905 m	9.25°
Viewing angle	15–30°	71/71	100.00%	0.903 m	22.32°
Viewing angle	30–45°	40/102	39.22%	0.904 m	37.45°
Speed	0.207 m/s	48/53	90.57%	0.862 m	17.76°
Speed	0.403 m/s	23/31	74.19%	0.858 m	15.53°
Speed	0.649 m/s	20/24	83.33%	0.889 m	19.98°
Speed	0.864 m/s	10/19	52.63%	0.899 m	21.44°
Speed	1.045 m/s	15/20	75.00%	0.925 m	23.51°

detector was highly reliable within the effective operating distance of approximately 0.50–1.50 m in the simulated setup.

This near-field limitation is geometric rather than a failure of the detector, and two mitigation strategies were already incorporated into the present system to address it. First, at the mission level, all AprilTag-triggered transitions in the proposed workflow are designed to occur within the validated 0.50–1.50 m operating band, so the robot is never required to detect a tag closer than 0.50 m and the near-field blind zone does not affect mission execution. Second, at the sensor level, the PTZ camera used for tag detection was configured with a wide horizontal field of view (1.8 rad, approximately 103°) so that the complete tag remains within the image frame over a larger portion of the near-field range; the 0.50 m boundary reported above therefore already reflects this wide field-of-view configuration and represents the residual geometric limit. Two further options can extend near-field robustness in future physical deployments: using a smaller tag or a multi-tag bundle so that at least one complete marker stays within the camera view at close range, and using the existing LiDAR range measurement as a complementary cue to anticipate the stopping point and pre-empt the transition before the robot enters the near-field blind zone.

In the viewing-angle evaluation, the detector achieved 100% success at 5–15° and 15–30°, showing that the system was robust when the tag was observed at moderate viewing angles. The success rate decreased at 0–5° and 30–45°, with values of 40.10% and 39.22%, respectively. These results suggest that detection reliability was not determined by viewing angle alone, but also by camera framing and tag visibility during the recorded motion. Therefore, the viewing-angle result should be interpreted together with the mean camera-to-tag distance and the visibility condition during the test.

In the speed evaluation, the highest detection success rate was obtained at an absolute commanded speed of 0.207 m/s, with a success rate of 90.57%. At higher speeds, the success rate varied between 52.63% and 83.33%. The lowest success rate occurred at 0.864 m/s, while the success rate increased again to 75.00% at 1.045 m/s. This non-monotonic trend indicates that robot speed was not the only factor affecting detection performance; camera-to-tag distance and viewing angle also influenced the result. Nevertheless, compared with the static distance test, the speed evaluation showed lower and more variable success rates, indicating that robot motion reduced AprilTag detection stability.

Overall, the AprilTag evaluation confirms that the detector performed reliably when the marker was observed within an appropriate distance and viewing-angle range. The distance test showed near-perfect detection beyond 0.50 m, while the viewing-angle and speed tests revealed that detection performance became more sensitive when the tag visibility changed during robot motion. These findings indicate that AprilTag-based localization is suitable as a supporting reference in

the simulated robot system, but the marker should be placed within a practical viewing distance and should remain sufficiently visible to the side-mounted camera during robot motion.

3.2. LiDAR-based navigation evaluation

In the simulated greenhouse environment, the proposed LiDAR-based navigation controller was evaluated in five crop-row scenarios and one wall-corridor reference scenario. The crop rows represent navigation conditions with irregular plant boundaries, while the wall corridor provides a more structured reference case with continuous and well-defined side boundaries. For each scenario, the robot was tested in both forward and backward directions. One manually driven ground-truth (GT) trajectory was recorded for each case, followed by three autonomous repetitions using the proposed controller.

Before evaluation, the recorded trajectories were preprocessed to remove invalid samples, suppress discontinuities, trim stationary sections at the beginning and end of each run, and reduce high-frequency noise. To ensure a fair comparison, each autonomous trajectory was aligned to the same start and end points as the corresponding GT trajectory and resampled into an equal number of points along the normalized path progress. Therefore, the reported trajectory error represents the repeatability of the path shape relative to the GT reference, rather than differences caused by unequal recording duration or stopping position.

Fig. 14 shows the trajectory RMSE for all scenarios after preprocessing and start–end normalization. Overall, the robot followed the GT trajectory closely in both forward and backward directions. The RMSE remained below 0.012 m for all tested rows and wall-corridor cases. This indicates that the proposed navigation method produced repeatable path-following behavior across different simulated greenhouse row conditions.

A quantitative summary of the navigation performance is presented in Table 6. In the crop-row scenarios, the forward-direction RMSE ranged from 0.0040 m to 0.0111 m, while the backward-direction RMSE ranged from 0.0011 m to 0.0021 m. The backward runs generally showed lower trajectory RMSE than the forward runs after normalization. This suggests that, under the evaluated simulation conditions, the robot was able to reproduce the return path with very high consistency once the start and end points were standardized.

Among the crop rows, Row 5 forward produced the largest average RMSE, with a value of 0.0111 m. However, this error was still small relative to the navigation corridor length, indicating that the robot remained close to the desired trajectory. Row 3 backward, which previously showed a large deviation using the earlier GT reference, produced a low RMSE of 0.0013 m after replacing the GT trajectory with the corrected reference. This confirms that the previous large error was caused

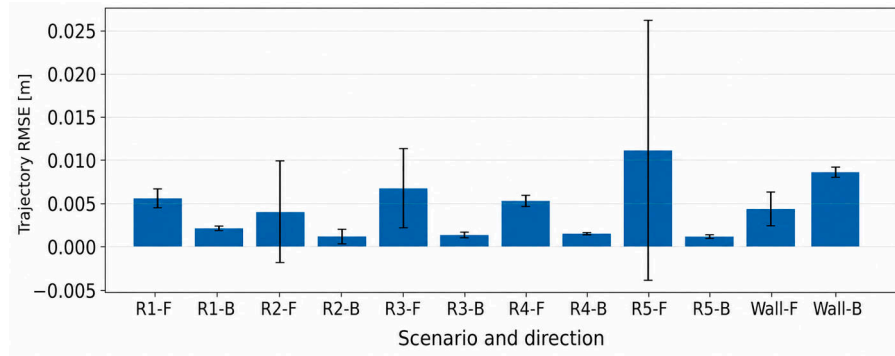


Fig. 14. Navigation trajectory RMSE across five greenhouse crop rows and one wall-corridor reference after trajectory preprocessing, start–end alignment, and normalized resampling. Labels R1–R5 indicate crop Rows 1–5, W indicates the wall-corridor reference, F indicates forward travel, and B indicates backward travel. Each bar represents the mean RMSE of three autonomous repetitions, while the error bar indicates the standard deviation.

Table 6

Navigation accuracy summary across five crop rows and the wall-corridor reference. Values are reported as mean $\pm$ standard deviation over three autonomous repetitions after preprocessing, start–end alignment, and normalized resampling.

Scenario	Direction	MAE (m)	RMSE (m)	Max error (m)	Heading MAE (°)
Row 1	Forward	0.005 $\pm$ 0.001	0.006 $\pm$ 0.001	0.008 $\pm$ 0.001	3.22 $\pm$ 0.29
Row 1	Backward	0.002 $\pm$ 0.000	0.002 $\pm$ 0.000	0.003 $\pm$ 0.000	3.51 $\pm$ 0.32
Row 2	Forward	0.004 $\pm$ 0.005	0.004 $\pm$ 0.006	0.007 $\pm$ 0.010	0.93 $\pm$ 0.05
Row 2	Backward	0.001 $\pm$ 0.001	0.001 $\pm$ 0.001	0.002 $\pm$ 0.002	0.83 $\pm$ 0.02
Row 3	Forward	0.006 $\pm$ 0.004	0.007 $\pm$ 0.005	0.012 $\pm$ 0.011	1.42 $\pm$ 0.09
Row 3	Backward	0.001 $\pm$ 0.000	0.001 $\pm$ 0.000	0.002 $\pm$ 0.001	0.60 $\pm$ 0.01
Row 4	Forward	0.005 $\pm$ 0.001	0.005 $\pm$ 0.001	0.007 $\pm$ 0.001	1.39 $\pm$ 0.01
Row 4	Backward	0.001 $\pm$ 0.000	0.002 $\pm$ 0.000	0.002 $\pm$ 0.000	2.20 $\pm$ 0.01
Row 5	Forward	0.010 $\pm$ 0.013	0.011 $\pm$ 0.015	0.019 $\pm$ 0.027	1.03 $\pm$ 0.17
Row 5	Backward	0.001 $\pm$ 0.000	0.001 $\pm$ 0.000	0.002 $\pm$ 0.001	1.68 $\pm$ 0.01
Wall	Forward	0.004 $\pm$ 0.001	0.004 $\pm$ 0.002	0.008 $\pm$ 0.005	2.36 $\pm$ 0.52
Wall	Backward	0.008 $\pm$ 0.001	0.009 $\pm$ 0.001	0.015 $\pm$ 0.001	4.29 $\pm$ 0.08

by a GT mismatch rather than instability of the autonomous navigation behavior.

The wall-corridor reference also showed stable trajectory tracking. The wall-forward RMSE was 0.0044 m, while the wall-backward RMSE was 0.0086 m. These values demonstrate that the controller can maintain a repeatable path when the side boundaries are continuous and geometrically regular. The slightly higher error in the wall-backward case is still within a small range and does not indicate loss of alignment.

Heading stability was also evaluated by comparing the yaw angle of each autonomous repetition with the corresponding GT yaw angle. The heading MAE remained below 4.30° in all scenarios. The lowest heading error was observed in Row 3 backward, with a value of 0.60°, while the largest heading error was observed in the wall-backward case, with a value of 4.29°. These results indicate that the robot maintained a stable orientation while performing lateral centering using its omnidirectional motion capability.

The consistently lower backward-direction RMSE (0.0011–0.0021 m, compared with 0.0040–0.0111 m in the forward direction) can be explained by three factors. First, in terms of heading-controller state, each backward pass began only after the forward pass had already centered the robot in the row and converged the lateral- and heading-control loops; the controller therefore started the return run from a well-aligned, low-transient condition, whereas the forward run included the initial convergence phase in which the robot first acquired the row center. This difference in starting condition is the main contributor to the lower backward error. Second, in terms of LiDAR point-cloud distribution, the left and right row boundaries are geometrically symmetric with respect to travel direction, and lateral centering is driven by the median-filtered left–right boundary distances

rather than by canopy tops or individual returns; the perceived control signal is therefore comparable in both directions and does not intrinsically favor one over the other. Third, and consequently, canopy occlusion has only a limited directional effect on lateral centering, since the filtered lateral boundaries remain observable even when parts of the upper canopy are occluded. It should also be noted that part of the reduction is associated with the start–end alignment and normalized resampling applied during trajectory post-processing, which standardizes the return path; the backward advantage should therefore be interpreted as a combination of this favorable starting condition and the normalization procedure, rather than as a purely physical gain in navigation accuracy.

Overall, the verification results show that the proposed LiDAR-based row-centering controller achieved accurate and repeatable navigation in the simulated greenhouse environment. The combination of longitudinal motion, lateral centering, and heading-hold control allowed the robot to follow the desired corridor path without relying solely on yaw correction. The low trajectory RMSE across both crop-row and wall-corridor scenarios confirms that the controller can maintain stable row alignment under the evaluated simulation conditions. The remaining differences among rows are mainly attributed to variations in local boundary geometry and the distribution of LiDAR returns from plant structures.

3.2.1. AprilTag-guided row navigation mission evaluation

After separately confirming the performance of the AprilTag detection module and the LiDAR-based row-centering controller in the previous subsections, the complete AprilTag-guided row navigation mission was evaluated according to the overall navigation framework described in Subsection 2.5.1. The mission was tested at commanded longitudi-

nal speeds of 0.2, 0.4, 0.6, 0.8, and 1.0 m/s. Each speed condition was repeated three times using the same greenhouse layout and mission sequence. During each autonomous run, the robot performed forward and backward in-row traversal, AprilTag-triggered stopping, PTZ-camera re-orientation, lateral transition to the adjacent crop row, and final mission termination.

A ground-truth (GT) reference trajectory was obtained by manually driving the robot through the same multi-row route at approximately 0.5 m/s. During the GT run, AprilTag-based mission control was disabled, and the PTZ camera and camera slider were not actuated. Therefore, the GT trajectory represents the desired geometric route used to evaluate the consistency of the autonomous mission trajectories.

Before evaluation, invalid odometry samples and stationary sections at the beginning and end of each run were removed. Each autonomous trajectory was translated to the same initial position as the GT trajectory. Trajectory samples extending beyond the total GT path length were removed so that all comparisons covered the same mission route.

Trajectory accuracy was evaluated using the shortest Euclidean distance between each autonomous trajectory point and the GT path. Let $\mathbf{p}_{a,i}$ denote the i th autonomous position and let $\mathcal{P}_g$ denote the set of GT trajectory points. The cross-track error was defined as

$$e_i = \min_{\mathbf{p}_{g,j} \in \mathcal{P}_g} \|\mathbf{p}_{a,i} - \mathbf{p}_{g,j}\|_2. \quad (24)$$

The mean absolute error (MAE), root mean square error (RMSE), 95th-percentile error, and maximum error were then calculated as

$$\text{MAE} = \frac{1}{N} \sum_{i=1}^N |e_i|, \quad (25)$$

$$\text{RMSE} = \sqrt{\frac{1}{N} \sum_{i=1}^N e_i^2}, \quad (26)$$

$$95\text{th percentile} = \text{percentile}_{95}(e_1, e_2, \dots, e_N), \quad \text{and} \quad (27)$$

$$e_{\max} = \max_{1 \leq i \leq N} (e_i). \quad (28)$$

where N is the number of valid autonomous trajectory samples. The MAE describes the average cross-track deviation, whereas the RMSE gives greater weight to larger local deviations. The 95th-percentile error represents the upper bound of the error for most of the trajectory, while the maximum error represents the largest isolated deviation observed during a run.

Fig. 15 compares the three autonomous repetitions at each commanded speed with the manually driven GT trajectory. All speed conditions reproduced the same overall multi-row navigation pattern, consisting of five longitudinal row segments connected by lateral transition movements. The close overlap between the autonomous and GT trajectories indicates that the proposed mission controller maintained the intended route over the full navigation sequence.

The quantitative trajectory results are summarized in Table 7. The lowest average deviation was obtained at 0.2 m/s, with an MAE of 0.045 ± 0.003 m and an RMSE of 0.064 ± 0.002 m. At 0.4 m/s, the MAE and RMSE increased slightly to 0.049 ± 0.017 m and 0.076 ± 0.026 m, respectively. The largest mean MAE was observed at 0.6 m/s, although it remained below 0.06 m.

Across all speed conditions, the mean cross-track error remained between 0.045 and 0.056 m. This relatively narrow range indicates that increasing the commanded speed did not substantially alter the overall geometric path followed by the robot. The RMSE values remained below 0.10 m for all tested speeds, showing that most trajectory samples remained close to the manually driven reference route.

The 95th-percentile error showed a greater speed-dependent variation than the MAE. It increased from 0.072 ± 0.004 m at 0.2 m/s to 0.149 ± 0.038 m at 0.8 m/s, before decreasing to 0.123 ± 0.041 m at 1.0 m/s. This result suggests that higher-speed runs produced larger local deviations during a limited portion of the mission, even though the average cross-track error remained small.

The maximum errors were considerably larger than the MAE and 95th-percentile values. As shown in Fig. 15, these isolated deviations occurred primarily near the lower ends of the longitudinal paths and during lateral row-transition movements. These sections required stopping, AprilTag confirmation, switching between navigation states, and

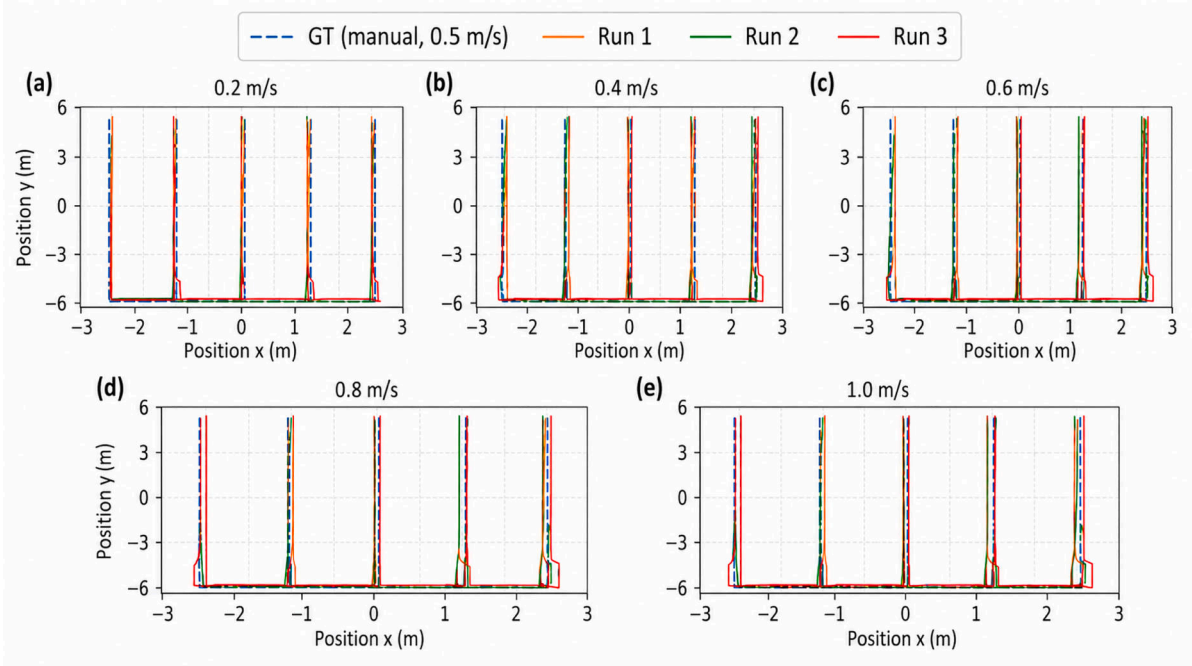


Fig. 15. Comparison between the manually driven ground-truth trajectory and the three autonomous AprilTag-guided mission runs at commanded speeds of (a) 0.2 m/s, (b) 0.4 m/s, (c) 0.6 m/s, (d) 0.8 m/s, and (e) 1.0 m/s. The GT trajectory was recorded at approximately 0.5 m/s without AprilTag-based mission control or PTZ-camera and slider actuation.

Table 7
Trajectory error metrics across different navigation speeds.

Speed (m/s)	Runs	MAE (m)	RMSE (m)	95th-percentile error (m)	Maximum error (m)
0.2	3	0.045 ± 0.003	0.064 ± 0.002	0.072 ± 0.004	0.815 ± 0.007
0.4	3	0.049 ± 0.017	0.076 ± 0.026	0.099 ± 0.012	0.914 ± 0.465
0.6	3	0.056 ± 0.011	0.094 ± 0.028	0.107 ± 0.011	0.827 ± 0.408
0.8	3	0.053 ± 0.011	0.091 ± 0.004	0.149 ± 0.038	0.825 ± 0.047
1.0	3	0.051 ± 0.004	0.092 ± 0.003	0.123 ± 0.041	0.841 ± 0.015

lateral alignment with the next row. Consequently, the maximum error reflects short local transition effects rather than persistent deviation during in-row traversal.

The large standard deviation of the maximum error at 0.4 and 0.6 m/s indicates that the magnitude of these isolated transition deviations varied among repetitions. In contrast, the relatively small standard deviations of the MAE values show that the overall trajectory-following behavior was repeatable across runs.

Overall, the results demonstrate that the proposed AprilTag-guided mission controller successfully reproduced the intended multi-row route at all evaluated speeds. The 0.2 m/s condition provided the highest trajectory accuracy, while speeds from 0.4 to 1.0 m/s produced only a modest increase in average cross-track error. Therefore, the selection of an operating speed may be based on the required balance between navigation accuracy, mission duration, and greenhouse monitoring efficiency.

3.3. Selection of a base model for ripeness-class fruit segmentation

Before evaluating fruit counting performance, a base segmentation model was selected for ripeness-class fruit segmentation. In this study, the model was required to localize individual cherry tomato fruits, generate instance masks, and assign each fruit instance to one of three ripeness classes: green, orange, or red. Therefore, the model selection process compared YOLOv8-seg and YOLOv11-seg models with three model scales: nano (n), small (s), and medium (m). Each model was trained using both non-augmented and augmented datasets.

The selection was not based only on the highest segmentation mAP value. Since the selected model was used in the subsequent ByteTrack-based fruit-counting evaluation, the model also needed to provide stable instance-level detections, balanced precision–recall behavior, and sufficient processing speed for video-based robot perception. In this context, false-positive detections may lead to overcounting, while missed detections may lead to undercounting. Therefore, mask precision, mask re-

call, mask F1-score, mask mAP50–95, bounding-box mAP50–95, mean inference time, and pipeline FPS were considered together.

Table 8 summarizes the validation performance and video-based inference speed of all trained models. The validation metrics were obtained from the best epoch selected based on mask mAP50–95, while the speed metrics were calculated from robot-camera video inference. The pipeline FPS represents the mean end-to-end throughput, including preprocessing, inference, and postprocessing.

Fig. 16 compares the mask mAP50–95 values among all model configurations. The augmented YOLOv11m-seg model achieved the highest mask mAP50–95 value of 0.416. However, this model also produced the lowest mask precision among all configurations, with a precision value of 0.646, and a relatively low mask F1-score of 0.706. In addition, its bounding-box mAP50–95 was only 0.336. These results indicate that although the model obtained the highest mask mAP50–95, its detection behavior was less balanced for downstream tracking-based counting.

Fig. 17 shows the speed–accuracy trade-off using pipeline FPS and mask F1-score. The nano-scale models generally provided higher processing speed, but their segmentation performance was lower than that of the small and medium models. The medium-scale models achieved stronger mask-level performance while still maintaining real-time processing capability. Among all candidates, the non-augmented YOLOv8m-seg model achieved the highest mask F1-score of 0.780, with mask precision of 0.790, mask recall of 0.770, mask mAP50–95 of 0.374, bounding-box mAP50–95 of 0.651, and pipeline throughput of 54.10 FPS.

Although augmented models and YOLOv11 variants achieved strong performance in several metrics, the final model selection prioritized the stability required for tracking-based counting rather than the highest individual metric. In particular, the augmented YOLOv11m-seg model was not selected despite having the highest mask mAP50–95 because its low precision and F1-score could increase the risk of unstable detections and overcounting. Based on the overall balance between mask segmentation quality, precision–recall behavior, localization performance, and real-

Table 8
Model selection results for cherry tomato ripeness-class fruit segmentation. Validation metrics were calculated at the best epoch selected by mask mAP50–95. Inference speed was averaged across robot-camera videos. The selected base model for the subsequent fruit-counting evaluation is marked with an asterisk (*).

Training	Model	P_M	R_M	$F1_M$	mAP50–95 _M	mAP50–95 _B	Time (ms)	FPS
Non-aug.	YOLOv8n	0.780	0.728	0.753	0.330	0.619	10.93	56.72
Non-aug.	YOLOv8s	0.786	0.770	0.778	0.368	0.655	12.06	53.81
Non-aug.	YOLOv8m*	0.790	0.770	0.780	0.374	0.651	13.11	54.10
Non-aug.	YOLOv11n	0.756	0.768	0.762	0.356	0.640	11.97	55.89
Non-aug.	YOLOv11s	0.792	0.759	0.775	0.372	0.649	13.61	49.89
Non-aug.	YOLOv11m	0.786	0.768	0.777	0.382	0.652	16.75	43.35
Aug.	YOLOv8n	0.786	0.759	0.772	0.363	0.643	11.79	57.21
Aug.	YOLOv8s	0.705	0.727	0.716	0.392	0.362	12.45	55.25
Aug.	YOLOv8m	0.772	0.779	0.775	0.377	0.670	16.25	45.42
Aug.	YOLOv11n	0.776	0.767	0.771	0.360	0.636	11.02	60.75
Aug.	YOLOv11s	0.795	0.757	0.776	0.376	0.594	12.35	55.86
Aug.	YOLOv11m	0.646	0.779	0.706	0.416	0.336	12.33	58.75

* Selected base model for the subsequent fruit-counting evaluation. P_M , R_M , $F1_M$, and mAP50–95_M denote mask precision, recall, F1-score, and mask mAP50–95, respectively. mAP50–95_B denotes bounding-box mAP50–95. Time indicates mean inference time per frame, while FPS indicates mean end-to-end pipeline throughput.

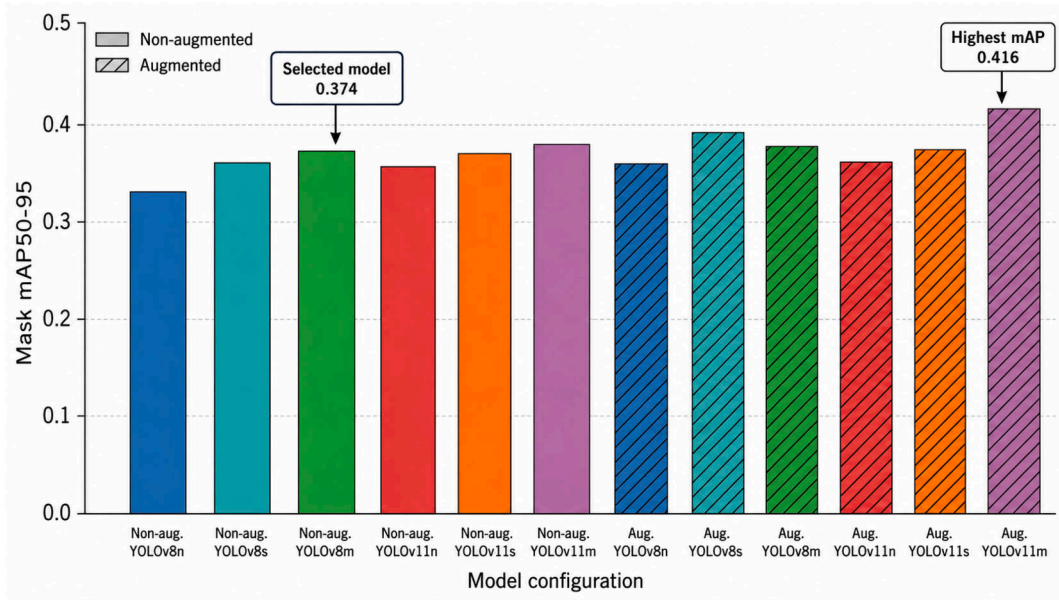


Fig. 16. Validation mask mAP50–95 comparison of YOLOv8-seg and YOLOv11-seg models for cherry tomato ripeness-class fruit segmentation under non-augmented and augmented training settings.

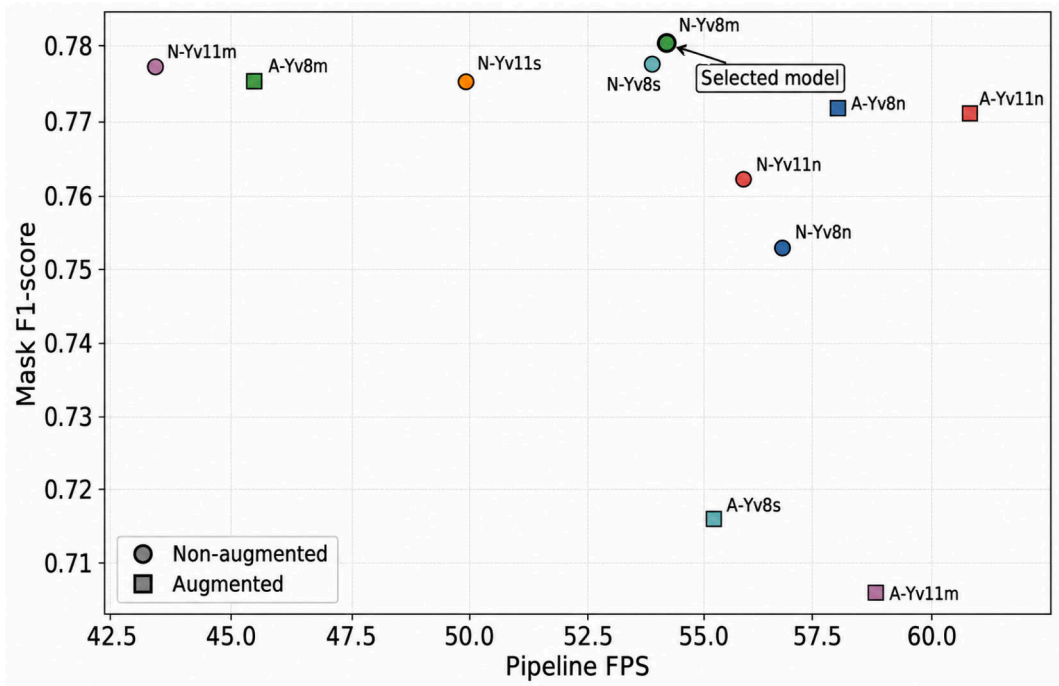


Fig. 17. Speed–accuracy trade-off of YOLO segmentation models for ripeness-class fruit segmentation. The x-axis shows mean pipeline FPS from robot-camera videos, while the y-axis shows mask F1-score. Circles indicate non-augmented models, and squares indicate augmented models.

time processing speed, the non-augmented YOLOv8m-seg model was selected as the base model for the subsequent fruit-counting evaluation.

3.4. Fruit counting performance and qualitative error analysis

Fruit-counting performance was evaluated at the individual-plant level using all 126 plants distributed across six greenhouse rows. Each plant was evaluated under a static-image condition and five robot traveling speeds ranging from 0.2 to 1.0 m/s. The same ground-truth plant community was used for all acquisition conditions and contained 1320 fruits in total, consisting of 828 red, 336 orange, and 156 green fruits.

Table 9 summarizes the counting performance under each acquisition condition. The best overall performance was obtained at 0.2 m/s, at which the system counted 1189 fruits, corresponding to 90.1% of the ground-truth total. At this speed, the plant-level MAE and RMSE were 2.02 and 4.16 fruits, respectively, and the count-level F1-score reached 0.898. The 0.2 m/s condition also outperformed the static-image condition, which recovered 81.2% of the ground-truth count and produced an F1-score of 0.822. This result suggests that slow camera motion provided temporal redundancy and multiple views of each plant, allowing some fruits that were not visible or confidently detected in a single static image to be recovered in subsequent frames.

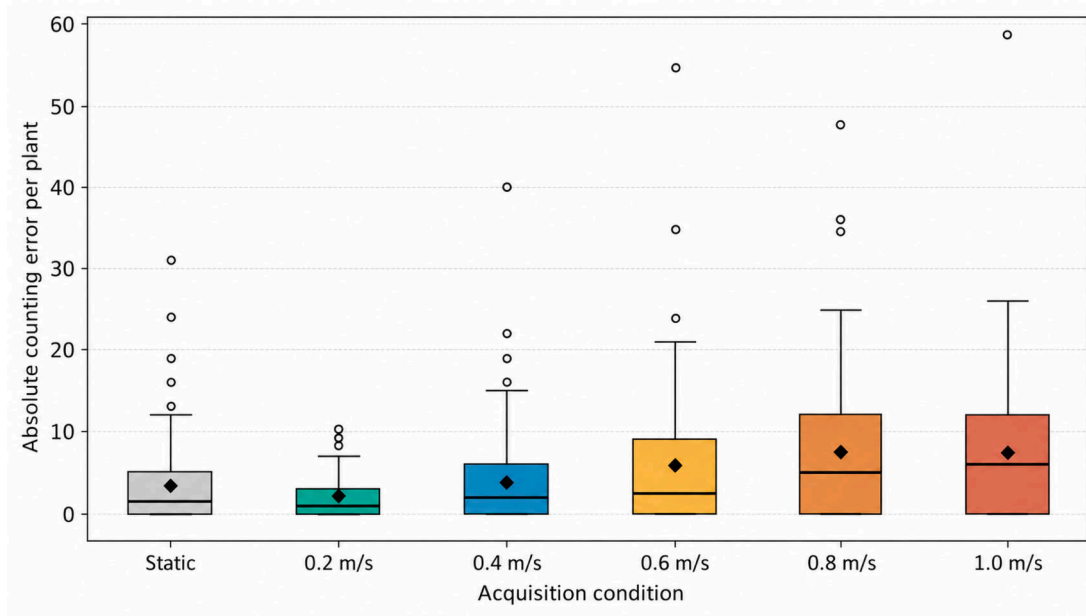


Fig. 18. Distribution of plant-level absolute counting errors for 126 plants under static and moving acquisition conditions. Each box represents the interquartile range, the horizontal black line indicates the median, and the black diamond indicates the mean. The whiskers represent the non-outlier range, while the open circles denote outlying plant-level errors. The static condition is shown in gray, the selected 0.2 m/s condition in green, the 0.4 m/s condition in blue, and the higher-speed conditions in progressively warmer colors. The lowest overall error distribution was obtained at 0.2 m/s, whereas error magnitude and variability increased at higher traveling speeds. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Table 9

Plant-level fruit-counting performance under static and moving acquisition conditions. Pred./GT denotes the percentage of the ground-truth fruit count recovered by the system. Precision, recall, and F1 are count-level metrics derived from plant-level counts and should not be interpreted as object-level detection or multi-object-tracking metrics.

Condition	Pred./GT (%)	MAE	RMSE	Precision	Recall	F1
Static	81.2	3.38	6.13	0.917	0.745	0.822
0.2 m/s	90.1	2.02	4.16	0.948	0.854	0.898
0.4 m/s	74.8	3.78	6.71	0.927	0.694	0.794
0.6 m/s	45.6	5.89	10.09	0.980	0.447	0.614
0.8 m/s	40.7	7.53	11.70	0.845	0.344	0.489
1.0 m/s	33.2	7.27	11.39	0.961	0.319	0.479

Fig. 18 presents the distribution of plant-level absolute counting errors. Unlike Table 9, which reports aggregate performance metrics, the figure shows the variability among individual plants. The 0.2 m/s condition exhibited the lowest central error and a comparatively narrow error distribution, whereas both error magnitude and dispersion increased at higher traveling speeds.

A Friedman test applied to the matched plant-level absolute errors confirmed a significant effect of acquisition condition ($\chi^2(5) = 162.95$, $p < 0.001$, Kendall's $W = 0.259$). Holm-adjusted pairwise comparisons showed that the 0.2 m/s condition produced significantly lower absolute errors than the static condition ($p = 0.040$) and all higher moving speeds ($p < 0.001$). No significant difference was found between the static and 0.4 m/s conditions ($p = 0.158$). These results support the selection of 0.2 m/s as the preferred speed for accurate counting and suggest that approximately 0.4 m/s represents the practical upper operating boundary for the present perception and tracking configuration.

The reduction in performance at higher speeds was driven primarily by missed fruits rather than excessive counting. At 0.6 and 1.0 m/s, count-level precision remained relatively high at 0.980 and 0.961, respectively, whereas recall decreased to 0.447 and 0.319. This indicates that many fruits were not detected or maintained as stable tracks for sufficient time to enter the counting region. Shorter observation duration, partial fruit visibility, and obstruction by the leaf canopy likely

contributed to the observed undercounting. Because these factors affect detection and tracking continuity, they cannot be fully corrected by the counting logic after a fruit has disappeared from the camera view.

The canonical-ID diagnostic logs showed that raw ByteTrack identities were frequently fragmented during video processing. Among the counted canonical tracks, 71.2% were associated with more than one raw tracker ID, and 62.0% underwent at least one relinking event after their initial count. Maintaining the counted state at the canonical-ID level was therefore important. When ByteTrack assigned a new raw ID to a previously counted fruit, the relinked track retained the same canonical identity and was not counted again. The canonical-ID layer consequently reduced the risk of duplicate counting caused by temporary tracking interruptions. However, these diagnostic results should not be interpreted as a standard multi-object-tracking evaluation because frame-level ground-truth identities were not available.

Ripeness-class predictions were also temporally inconsistent for some tracks. Approximately 26.1% of counted canonical tracks received more than one ripeness-class label during their lifetime, while red-orange alternation occurred in 18.4% of the counted tracks. This behavior is consistent with the visual similarity of fruits near the orange-to-red transition. Nevertheless, the available plant-level ground truth cannot determine whether every class-count difference resulted from a true class switch, a missed fruit, or an additional detection. The quantitative results were therefore complemented by representative visual cases obtained at the selected speed of 0.2 m/s.

Fig. 19 presents three correctly counted cases, one for each ripeness class, together with representative class-confusion and canopy-obstruction cases. Row 5 Plant 7, Row 5 Plant 11, and Row 2 Plant 15 were selected as correct-counting examples for red, green, and orange fruits, respectively. For these plants, the predicted count for the selected ripeness class matched the corresponding ground-truth count.

Row 1 Plant 8 was selected as the red-orange class-confusion example. Its ground truth contained 12 orange fruits and no red fruits, whereas the predicted result contained 10 orange fruits and one red fruit. Visual inspection indicated that an orange fruit was assigned to the red class, while an additional orange fruit was not counted. Row 6

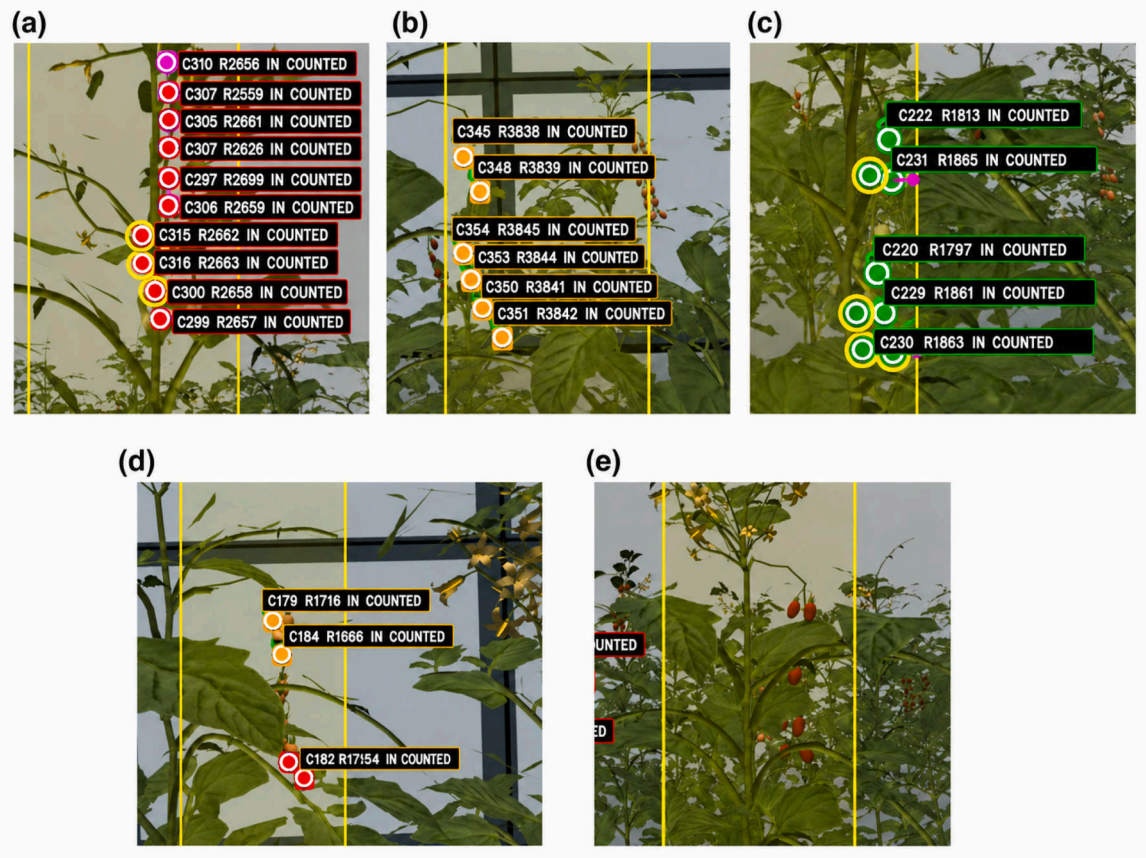


Fig. 19. Representative fruit-counting cases at 0.2 m/s. Each detection is labeled with its canonical identity (C), its current raw ByteTrack identity (R), and its counting status; yellow vertical lines denote the counting region of interest (ROI). (a) Correct red-fruit counting (Row 5, Plant 7); (b) correct orange-fruit counting (Row 2, Plant 15); (c) correct green-fruit counting (Row 5, Plant 11), where the predicted count for the selected ripeness class matched the ground truth. (d) Red-orange class-confusion case (Row 1, Plant 8): the ground truth contained 12 orange and no red fruits, whereas 10 orange and one red fruit were predicted, as one orange fruit was assigned to the red class and another orange fruit was not counted. (e) Canopy-obstruction case (Row 6, Plant 1): 9 of the 12 ground-truth red fruits were counted, with the missed fruits partially obscured by the leaf canopy. (For interpretation of the references to color in this figure legend, the reader is referred to the web version of this article.)

Plant 1 was selected as the canopy-obstruction example. The system counted 9 of the 12 ground-truth red fruits, with the missed fruits being partially obscured by the leaf canopy.

3.5. Simulation-to-real gap, limitations, and planned physical validation

The Gazebo environment provided a controlled and repeatable platform for integrating the navigation, perception, tracking, and fruit-counting modules. Procedural generation also enabled different plant and fruit distributions to be evaluated while retaining exact plant-level ground truth. Nevertheless, the reported results represent controlled-simulation performance and should not be interpreted as direct estimates of performance in a real greenhouse [39].

Several differences between the simulated and real environments may affect system performance. First, simulated illumination remained relatively stable, whereas real greenhouses exhibit sunlight variation, shadows, reflections, glare, and changes in camera exposure. These effects may reduce segmentation confidence and increase confusion between visually similar orange and red fruits. Second, the simulated plants and greenhouse structures used a limited set of meshes and textures and therefore did not reproduce the full variability of real fruit shape, surface appearance, leaf condition, background clutter, and material reflectance.

The current LiDAR simulation also did not include a sensor-specific noise and measurement-dropout model. Real LiDAR measurements may contain range uncertainty, missing returns, outliers, and unstable re-

sponses from thin leaves, stems, and reflective greenhouse structures. Consequently, the simulated crop-row boundaries were likely more stable than those expected during physical operation, and the reported navigation performance may represent an optimistic condition. In addition, the simulated plants were static, whereas real leaves and fruit clusters may move because of airflow, vibration, irrigation, or robot contact. Such motion may alter LiDAR boundary measurements, temporarily occlude fruits, interrupt detections, and increase tracker-ID fragmentation.

The AprilTag-guided mission also assumes that the required markers remain visible, clean, and correctly positioned. In a real greenhouse, tags may be partially or fully occluded by leaves, stems, fruit clusters, tools, or workers. Dust, water droplets, mud, condensation, fading, and physical damage may also reduce image contrast and prevent reliable tag recognition. Variations in viewing angle, camera exposure, motion blur, and tag displacement may further delay or suppress the mission-state transitions that depend on tag detection. Therefore, practical deployment should include redundant markers, improved tag placement, periodic tag maintenance, and fallback behavior when an expected tag cannot be detected.

Another limitation is that the current navigation algorithm does not explicitly handle dynamic obstacles that may interrupt the mission. The simulated environment did not include moving workers, carts, tools, animals, or other robots crossing the robot path. The present controller primarily maintains row centering and executes predefined AprilTag-triggered transitions; it does not perform dynamic-obstacle prediction, collision avoidance, or autonomous path replanning. In physical opera-

tion, a temporary obstruction could block the LiDAR field of view, conceal an AprilTag, or interfere with lateral row-transition motion. Future implementations should therefore incorporate safety stopping, obstacle classification, local replanning, and mission-resumption logic before real greenhouse deployment.

A further limitation concerns the spatial extent of the counting region of interest (ROI). The ROI restricts counting to a lateral band around the current row, and the canonical-identity layer prevents the same tracked fruit from being counted more than once over time. However, neither mechanism explicitly rejects fruits that physically belong to an adjacent row but happen to fall within the ROI in the image plane. In dense plantings, a robust detector may therefore register fruits from a neighboring row and inflate the count for the current row. The present ROI plus canonical-identity strategy thus suppresses temporal duplicate counting, but does not fully eliminate spatial leakage between adjacent rows. A depth-based validation step, described in the planned physical-validation study below, is proposed to address this leakage using the depth-sensing capability already present on the robot.

These limitations may influence the absolute navigation, segmentation, and counting performance, although the controlled simulation remains useful for comparing acquisition conditions and identifying system behavior. In particular, the preferred counting speed of 0.2 m/s and the approximate upper operating boundary of 0.4 m/s are specific to the present camera, LiDAR, plant model, tracking configuration, and simulated mission environment, and require validation under real greenhouse illumination, plant motion, marker degradation, sensor noise, and dynamic-obstacle conditions.

The choice of simulation platform also shapes how well these results transfer to reality. Gazebo Classic, used in this study, provides mature ROS and ROS 2 integration and a lightweight footprint that runs on modest hardware, but it offers only basic rendering and idealized sensor models, and it has reached end-of-life. NVIDIA Isaac Sim, by contrast, provides GPU-accelerated PhysX physics, RTX ray-traced photorealistic rendering, and higher-fidelity camera and LiDAR sensor models with configurable noise, together with built-in domain-randomization and synthetic-data generation (Replicator); these capabilities come at the cost of substantially higher GPU requirements. For the present workflow, the most relevant expected benefit is more realistic lighting, occlusion, and sensor noise, which should narrow the perception sim-to-real gap and improve the transferability of the segmentation, tracking, and counting results, while the mission-control and LiDAR-centering logic would remain largely unchanged [39,40].

To verify that these simulation results transfer to real conditions, a physical-validation study is planned as the immediate next phase of this work. It will use a wheeled Mecanum robot carrying the same sensor suite modeled here—a 3D LiDAR, a PTZ-RGB camera, and a front RGB camera—and will run the same ROS 2 nodes used in the virtual workflow, so that only the environment and physical sensors differ. The validation follows a staged protocol in which each subsystem is verified before the integrated mission is tested:

1. *Sensor calibration and bring-up*: camera intrinsic and extrinsic calibration, LiDAR calibration, AprilTag pose-estimation checks across the operating distances, and LiDAR-odometry time synchronization.
2. *Single-row navigation*: LiDAR row-centering along one real crop row, quantified by cross-track RMSE and heading error against a surveyed reference path at the same commanded speeds used in simulation.
3. *Full multi-row mission*: end-to-end AprilTag-guided forward and backward traversal, PTZ reorientation, and lateral row transitions across several rows, assessed by mission-completion rate and transition reliability.
4. *Fruit counting*: class-wise cherry-tomato counting against manual per-plant ground truth, evaluated with the same plant-level MAE and RMSE, recovery rate, and count-level precision, recall, and F1-score reported here, and complemented by standard multi-object-tracking metrics (MOTA, IDF1, and ID switches).

5. *Depth-based count validation*: estimating the depth of the row currently under observation from the on-board LiDAR or depth camera, deriving the expected fruit-depth distribution for that row, and rejecting detections whose estimated depth deviates beyond a threshold, so that fruits belonging to adjacent rows are excluded from the count. Combined with the lateral ROI and canonical-identity tracking, this depth gate targets spatial neighbor-row leakage in addition to temporal duplicates, and is expected to improve counting reliability in dense multi-row plantings.

Because the same metrics are reported in both domains, this protocol will allow a direct simulation-to-real comparison and quantify the reality gap for navigation and counting. Particular attention will be given to the counting-speed guidance derived in simulation (preferred 0.2 m/s, approximate upper boundary 0.4 m/s), which will be re-measured under real illumination, plant motion, marker degradation, sensor noise, and dynamic obstacles to confirm or revise the recommended operating range.

4. Conclusion

This study developed and evaluated a simulation-driven ROS 2–Gazebo workflow for integrating greenhouse robot navigation, PTZ-RGB fruit perception, temporal tracking, and ripeness-specific cherry-tomato counting. The main conclusions are summarized as follows:

1. The developed virtual greenhouse provided a controlled and repeatable environment for evaluating the complete navigation and perception workflow before physical deployment, with procedural plant generation enabling testing under heterogeneous canopy structures and fruit distributions while retaining exact plant-level ground truth.
2. The LiDAR-based row-centering controller achieved accurate and repeatable navigation in both directions, with trajectory RMSE below 0.012 m across all crop-row and wall-corridor scenarios. Backward traversal showed lower RMSE (0.0011–0.0021 m) than forward traversal (0.0040–0.0111 m), because each backward pass started from an already-centered, converged state, whereas the forward runs included the initial row-acquisition transient; the largest error occurred in the forward direction (Row 5 forward, 0.0111 m), and part of this difference also reflects the start–end normalization applied during trajectory post-processing. The remaining differences among rows are mainly attributed to variations in local boundary geometry and LiDAR return distribution.
3. The selected YOLOv8m-seg model, combined with ByteTrack and canonical-ID-based counting, enabled class-wise counting of green, orange, and red cherry tomatoes, with the canonical-ID layer preserving the counted state when raw tracker identities changed and thereby reducing duplicate counting after temporary tracking interruptions.
4. Robot traveling speed significantly influenced counting performance: the best result was obtained at 0.2 m/s (90.1% of the ground-truth count; count-level F1-score 0.898), remained comparatively acceptable at 0.4 m/s, and declined substantially from 0.6 m/s onward. The main errors arose from leaf-canopy occlusion, limited observation duration at higher speeds, and classification inconsistency between adjacent orange and red ripeness stages, showing that counting performance depends jointly on robot motion, canopy structure, segmentation quality, and tracking stability.
5. The main research significance of this work is the demonstration that simulation can serve not only as a pre-deployment testing platform, but also as a diagnostic tool for identifying navigation, perception, tracking, and counting limitations under controlled and repeatable conditions.

The simulation-to-real limitations of the present study have been discussed in Section 3.5. Future work will first improve the simulation fidelity before conducting physical validation. Higher-fidelity platforms,

such as NVIDIA Isaac Sim, may be used to provide more realistic lighting, materials, sensor behavior, and robot–environment interaction through GPU-accelerated physics, photorealistic RTX rendering, and higher-fidelity camera and LiDAR sensor models with built-in domain-randomization tools [40], which are expected to improve perception sim-to-real transfer relative to the Gazebo Classic environment used here. The procedural plant generator will also be extended to represent greater variation in plant height, canopy density, leaf size and orientation, stem structure, fruit size, shape, color, reflectance, ripeness distribution, and occlusion level. Dynamic leaf and fruit motion, camera noise, motion blur, LiDAR uncertainty, and measurement dropout will be incorporated to support domain randomization and more demanding robustness tests. The complete workflow will subsequently be validated using a physical robot in real greenhouse rows, with LiDAR–vision–odometry fusion and real image sequences containing persistent fruit identities to enable object-level evaluation using standard multi-object-tracking metrics (e.g., MOTA, IDF1, and ID switches) alongside the count-level precision and recall used in this study.

CRedit authorship contribution statement

Sutan Muhamad Sadam Awal: Writing – review & editing, Writing – original draft, Visualization, Validation, Software, Resources, Methodology, Investigation, Formal analysis, Data curation, Conceptualization; **Yamato Sakurai:** Writing – original draft, Visualization, Investigation, Formal analysis, Data curation, Conceptualization; **Dong Thanh Pham:** Writing – review & editing, Validation, Supervision, Software, Investigation, Formal analysis; **Koichi Nomura:** Validation, Supervision, Project administration, Funding acquisition; **Masaharu Kitano:** Validation, Supervision, Project administration, Funding acquisition; **Daisuke Yasutake:** Validation, Supervision; **Muneshi Mitsuoka:** Writing – review & editing, Validation, Supervision, Formal analysis; **Takashi Okayasu:** Writing – review & editing, Validation, Supervision, Project administration, Investigation, Funding acquisition, Formal analysis, Data curation, Conceptualization.

Ethical statement

This study was conducted in compliance with all institutional and national guidelines. No ethical approval was required as this research did not involve humans, animals, or sensitive materials.

Declaration of generative AI and AI-assisted technologies in the manuscript preparation process

During the preparation of this work, the authors used ChatGPT by OpenAI for language refinement, coding assistance, and the preparation of selected conceptual schematic illustrations. The AI-assisted illustrations were limited to explanatory materials and were not used to generate, modify, or represent experimental data or research results. The authors reviewed, verified, and edited all AI-assisted content and take full responsibility for the content of the publication.

Declaration of competing interest

The authors declare the following financial interests/personal relationships which may be considered as potential competing interests:

Sutan Muhamad Sadam Awal reports financial support was provided by IoP (Internet of Plants) Collaborative Creation Center, Kochi University. Sutan Muhamad Sadam Awal reports a relationship with IoP (Internet of Plants) Collaborative Creation Center, Kochi University that includes: employment. If there are other authors, they declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Acknowledgments

This work was supported by Cabinet Office grant in aid “Evolution to Society 5.0 Agriculture Driven by IoP (Internet of Plants),” Japan.

Data availability

Data will be made available on request.

References

- [1] Food and Agriculture Organization of the United Nations, Employment indicators 2000–2022: October 2024 update, FAOSTAT Analytical Brief, Accessed 2026 (2024).
- [2] Food and Agriculture Organization of the United Nations, The State of Food and Agriculture 2022: Leveraging Automation in Agriculture for Transforming Agrifood Systems, FAO, Rome, 2022. <https://doi.org/10.4060/cb9479en>
- [3] Food and Agriculture Organization of the United Nations, Enabling inclusive agricultural automation, policy brief adapted from The State of Food and Agriculture 2022 (2022).
- [4] Ministry of Agriculture, Forestry and Fisheries, Statistical Survey on Farm Management, 2020, <https://www.maff.go.jp/e/data/stat/>, Accessed 2025.
- [5] C.W. Bac, E.J. van Henten, J. Hemming, Y. Edan, Harvesting robots for high-value crops: state-of-the-art review and challenges ahead, *J. Field Robot.* 31 (6) (2014) 888–911. <https://doi.org/10.1002/rob.21525>
- [6] N. Burgess, J.A. King, Navigation in virtual space: psychological and neural aspects, in: N.J. Smelser, P.B. Baltes (Eds.), *International Encyclopedia of the Social & Behavioral Sciences*, Pergamon, Oxford, 2001, pp. 10417–10422. <https://doi.org/10.1016/B0-08-043076-7/03564-6>
- [7] Gazebo Documentation, Gazebo simulator overview, 2021, <https://gazebo.sim.org>, Accessed 1 Apr 2025.
- [8] N. Koenig, A. Howard, Design and use paradigms for Gazebo, an open-source multi-robot simulator, in: 2004 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS) (IEEE Cat. No.04CH37566), vol. 3, 2004, pp. 2149–2154. <https://doi.org/10.1109/IROS.2004.1389727>
- [9] Z. Li, R. Xu, C. Li, L. Fu, Visual navigation and crop mapping of a phenotyping robot MARS-PhenoBot in simulation, *Smart Agric. Technol.* 11 (2025) 100910. <https://doi.org/10.1016/j.atech.2025.100910>
- [10] G. Betrò, S. Pascuzzi, F. Paciolla, A GNSS-free LiDAR-based navigation architecture for autonomous inter-row operation under sparse or absent vegetation conditions, *Smart Agric. Technol.* 14 (2026) 102106. <https://doi.org/10.1016/j.atech.2026.102106>
- [11] M. Martini, M. Ambrosio, A. Navone, B. Tuberga, M. Chiaberge, Enhancing visual autonomous navigation in row-based crops with effective synthetic data generation, *Precis. Agric.* 25 (2024) 2881–2902. <https://doi.org/10.1007/s11119-024-10157-6>
- [12] J. Iqbal, R. Xu, S. Sun, C. Li, Simulation of an autonomous mobile robot for LiDAR-based in-field phenotyping and navigation, *Robotics* 9 (2) (2020) 46. <https://doi.org/10.3390/robotics9020046>
- [13] R. Singh, L. Seneviratne, I. Hussain, A deep learning-based approach to strawberry grasping using a telescopic-link differential drive mobile robot in ROS-Gazebo for greenhouse digital twin environments, *IEEE Access* 13 (2025) 361–381. <https://doi.org/10.1109/ACCESS.2024.3520233>
- [14] J. Deng, S. Marri, J. Klein, W. Palubicki, S. Pirk, G. Chowdhary, D.L. Michels, Gazebo plants: simulating plant-robot interaction with cossarat rods, (2024). *ArXiv abs/2402.02570* <https://api.semanticscholar.org/CorpusID:267413073>.
- [15] A. Ivanovic, M. Polic, J. Tabak, M. Orsag, Render-in-the-loop aerial robotics simulator: case study on yield estimation in indoor agriculture, 2022 International Conference on Unmanned Aircraft Systems (ICUAS) (2022) 787–793. <https://api.semanticscholar.org/CorpusID:247188025>.
- [16] S. Jiang, P. Qi, L. Han, L. Liu, Y. Li, Z. Huang, Y. Liu, X. He, Navigation system for orchard spraying robot based on 3D LiDAR SLAM with NDT_ICP point cloud registration, *Comput. Electron. Agric.* 220 (2024) 108870. <https://doi.org/10.1016/j.compag.2024.108870>
- [17] T. Fujinaga, Autonomous navigation method for agricultural robots in high-bed cultivation environments, *Comput. Electron. Agric.* 231 (2025) 110001. <https://doi.org/10.1016/j.compag.2025.110001>
- [18] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, W. Woodall, Robot operating system 2: design, architecture, and uses in the wild, *Sci. Robot.* 7 (66) (2022) eabm6074. <https://doi.org/10.1126/scirobotics.abm6074>
- [19] AprilTag Documentation, AprilTag Technical Overview, 2022, <https://april.eecs.umich.edu>. Accessed 1 Apr 2025.
- [20] J. Redmon, S. Divvala, R. Girshick, A. Farhadi, You only look once: unified, real-time object detection, in: 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016, pp. 779–788. <https://doi.org/10.1109/CVPR.2016.91>
- [21] R. Khanam, M. Hussain, YOLOv11: an overview of the key architectural enhancements, (2024) *arXiv:2410.17725*. <https://api.semanticscholar.org/CorpusID:273532028>.
- [22] G. Jocher, J. Qiu, Ultralytics YOLO11, 2024, License: AGPL-3.0, <https://github.com/ultralytics/ultralytics>.
- [23] Viso.AI, YOLOv11: a new iteration of “you only look once”, 2024, <https://viso.ai/computer-vision/yolov11/>, Accessed 1 Apr 2025.

- [24] J. Wei, Y. Sun, L. Luo, L. Ni, M. Chen, M. You, Y. Mu, H. Gong, Tomato ripeness detection and fruit segmentation based on instance segmentation, *Front. Plant Sci.* 16 (2025) 1503256. <https://doi.org/10.3389/fpls.2025.1503256>
- [25] D. Song, P. Liu, Y. Zhu, T. Li, K. Zhang, FGS-YOLOv8s-Seg: a lightweight and efficient instance segmentation model for detecting tomato maturity levels in greenhouse environments, *Agronomy* 15 (7) (2025) 1687. <https://doi.org/10.3390/agronomy15071687>
- [26] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, X. Wang, ByteTrack: multi-object tracking by associating every detection box, in: *European Conference on Computer Vision*, Springer, 2022, pp. 1–21.
- [27] X. Yang, Y. Gao, M. Yin, H. Li, Automatic apple detection and counting with AD-YOLO and MR-SORT, *Sensors* 24 (21) (2024) 7012. <https://doi.org/10.3390/s24217012>
- [28] P.K. Sekharamanthy, F. Melgani, J. Malacarne, R. Ricci, R. de Almeida Silva, J. Marcato Junior, A seamless deep learning approach for apple detection, depth estimation, and tracking using YOLO models enhanced by multi-Head attention mechanism, *Computers* 13 (3) (2024) 83. <https://doi.org/10.3390/computers13030083>
- [29] T. Jin, X. Han, P. Wang, Z. Zhang, J. Guo, F. Ding, Enhanced deep learning model for apple detection, localization, and counting in complex orchards for robotic arm-based harvesting, *Smart Agric. Technol.* 10 (2025) 100784. <https://doi.org/10.1016/j.atech.2025.100784>
- [30] Anonymous, Details omitted for double-anonymized reviewing, Details omitted for double-anonymized reviewing (2024).
- [31] J.P. Espejel Flores, A. Yilmaz, L.A. Soriano Avendaño, G. Cielniak, Comparative analysis of unity and Gazebo simulators for digital twins of robotic tomato harvesting scenarios, in: M.N. Huda, M. Wang, T. Kalganova (Eds.), *Towards Autonomous Robotic Systems*, Springer Nature Switzerland, Cham, 2025, pp. 15–27.
- [32] C.E. Shannon, A mathematical theory of communication, *Bell Syst. Tech. J.* 27 (3) (1948) 379–423. <https://doi.org/10.1002/j.1538-7305.1948.tb01338.x>
- [33] E.C. Pielou, The measurement of diversity in different types of biological collections, *J. Theor. Biol.* 13 (1966) 131–144. [https://doi.org/10.1016/0022-5193\(66\)90013-0](https://doi.org/10.1016/0022-5193(66)90013-0)
- [34] T.-Y. Lin, M. Maire, S. Belongie, J. Hays, P. Perona, D. Ramanan, P. Doll'ar, C.L. Zitnick, Microsoft COCO: common objects in context, in: *European Conference on Computer Vision*, Springer, 2014, pp. 740–755. https://doi.org/10.1007/978-3-319-10602-1_48
- [35] Ultralytics, YOLO performance metrics, 2024, Ultralytics Documentation, Accessed 2026.
- [36] Y. Zhang, P. Sun, Y. Jiang, D. Yu, F. Weng, Z. Yuan, P. Luo, W. Liu, X. Wang, ByteTrack: Multi-Object Tracking by Associating Every Detection Box, 2022, [arXiv:2110.06864](https://arxiv.org/abs/2110.06864) [cs.CV]
- [37] M. Friedman, The use of ranks to avoid the assumption of normality implicit in the analysis of variance, *J. Am. Stat. Assoc.* 32 (200) (1937) 675–701. <https://doi.org/10.1080/01621459.1937.10503522>
- [38] S. Holm, A simple sequentially rejective multiple test procedure, *Scand. J. Stat.* 6 (2) (1979) 65–70.
- [39] W. Zhao, J.P. Queralta, T. Westerlund, Sim-to-real transfer in deep reinforcement learning for robotics: a survey, in: *2020 IEEE Symposium Series on Computational Intelligence (SSCI)*, 2020, pp. 737–744. <https://doi.org/10.1109/SSCI47803.2020.9308468>
- [40] J. Tobin, R. Fong, A. Ray, J. Schneider, W. Zaremba, P. Abbeel, Domain randomization for transferring deep neural networks from simulation to the real world, in: *2017 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, 2017, pp. 23–30. <https://doi.org/10.1109/IROS.2017.8202133>